

Plan de Desarrollo Detallado: StatsPro Basketball

- **ID de referencia:** PRD-BSKT-2026-001
- **Estado:** Aprobado
- **Producto:** StatsPro Basketball
- **Ingeniería:** equipo de 2 ingenieros
- **Clasificación:** interno
- **Stack principal:** SQLite · Python/Pandas · Flet (UI, pendiente ADR-002)

Qué es este documento: transcripción completa y fusionada del PRD del proyecto — todas las historias de usuario, épicas e hitos, con sus criterios de aceptación, archivos a crear y funciones/métodos involucrados. Combina dos fuentes que viven en el submódulo `git docs/documentacion_app_estadistica/`

Donde una fuente tenía más detalle que la otra para la misma sección, se usó la más completa; donde ambas aportaban algo distinto, se fusionó. **No es un resumen** — la única compresión respecto a las fuentes originales es quitar el formato LaTeX (cajas de color, `tcolorbox`) y reemplazarlo por Markdown plano.

Al final del documento hay una sección **"Estado real del código vs. plan"** con los hallazgos e inconsistencias encontrados al hacer esta transcripción — ver sección 20.

Tabla de Contenidos

1. Descripción General del Producto
2. Perfiles de Usuario
3. Arquitectura de Datos
4. Arquitectura de Software (Clean + Hexagonal)
5. Acuerdo de Ingeniería y Estándares
6. Reglas de Negocio Consolidadas
7. Requisitos No Funcionales (NFR)
8. Registro de Decisiones Arquitectónicas (ADR)
9. Hito 1 — Núcleo de Datos e Interfaz CLI (v0.1)
10. Hito 2 — Motor de Ingesta y Análisis (v0.2)
11. Hito 3 — Visualización Pro y Reporting (v0.3)
12. Hito 4 — Interfaz Multiplataforma y Entrega (v1.0)
13. Definición de "Hecho" (DoD)
14. Catálogo Técnico de Criticidad
15. Proceso de Liberación de Versiones
16. Roadmap Futuro (Hitos 5–9, visión de producto)
17. Estructura de Repositorios
18. ADRs Pendientes — tabla de bloqueo por hito
19. Convenciones rápidas
20. Estado real del código vs. plan (hallazgos)

1. Descripción General del Producto

1.1 Planteamiento del problema

El análisis estadístico en el básquet formativo sufre una brecha crítica entre la recolección de datos y su utilidad táctica. Los entrenadores operan bajo alta presión donde la toma de decisiones basada en datos se ve limitada por herramientas fragmentadas.

#	Problema	Impacto Operativo	Línea Base
1	Carga manual ineficiente	Interferencia táctica; pérdida de foco durante el partido	>20 min/partido
2	Ceguera estadística	Decisiones por intuición, sin métricas avanzadas (PPP, EFF)	0% automatizado
3	Fragmentación de datos	Imposibilidad de seguimiento histórico o comparativo de rivales	Papel / Excel
4	Inestabilidad de red	Apps web fallan en estadios sin conectividad	100% online (competidores)
5	Rigidez de plataformas	El DT no puede integrar datos de Ges Deportivo fácilmente	Ingesta manual

Desglose adicional (basado en encuestas reales a DTs):

- **Interferencia en el juego:** la carga manual en tiempo real distrae al entrenador o requiere una persona dedicada exclusivamente a esa tarea.
- **Ritmo frenético:** la velocidad del básquet genera demoras entre la carga de una estadística y la siguiente.
- **Cálculo manual y falta de tendencias:** los DTs pierden tiempo pasando el boxscore a mano y calculando estadísticas avanzadas por su cuenta; no logran identificar numéricamente hacia dónde van las tendencias del juego.
- **Falta de centralización:** no hay un espacio virtual cómodo para el registro histórico, lo que dificulta comparar rivales o evaluar la evolución de los propios jugadores.
- **Herramientas inflexibles:** las apps existentes no se adaptan a los sistemas de competencia locales, y los datos de plataformas externas (Ges Deportivo) no se integran sin trabajo manual.

Línea base cuantificada: carga manual >20-40 min; 0% de automatización de importación Excel; 65% de los usuarios encuestados requieren uso offline; datos aislados por partido sin acumulación histórica.

1.2 Visión del producto

Desarrollar una aplicación multiplataforma (PC y Mobile) de ejecución local que centralice el conocimiento deportivo en un motor estadístico profesional, permitiendo una gestión integral **sin necesidad de internet**, con un flujo optimizado mediante la carga de planillas de "Ges Deportivo".

Pilares del producto:

1. **Accesibilidad y flexibilidad:** acceso desde cualquier dispositivo — cancha (celular/tablet) o casa (PC/notebook) para análisis más profundo.

- 2. **Gestión organizativa completa:** perfil de usuario, club, categorías, competencias y listas de buena fe.
- 3. **Ingesta de datos automatizada y manual:** además de la carga manual partido a partido, el software interpreta automáticamente planillas Excel de Ges Deportivo.
- 4. **Generación de estadísticas avanzadas:** motor de análisis que entrega resúmenes y estadísticas acumuladas, tradicionales y avanzadas, individuales y de equipo.
- 5. **Visualización y toma de decisiones:** comparación de equipos/jugadores, filtros, gráficos claros para identificar tendencias a corto, mediano y largo plazo.

1.3 Metas y no metas

En el alcance (v1.0):

- **Ingesta multimodal:**
 - *Automática:* carga de planillas Excel de Ges Deportivo.
 - *Manual:* formulario para carga post-partido (jugador por jugador).
- **Persistencia local:** base de datos SQLite única por instalación de usuario.
- **Motor estadístico:** análisis de tendencias, PPP, estadística avanzada y eficiencia con Pandas.
- **Multiplataforma:** ejecución en Windows, Linux, macOS y dispositivos móviles.

Fuera de alcance (v1.0):

- **Carga en vivo:** toma de datos en tiempo real durante el partido — se posterga a versiones post-estables (ver Hito 7 en el roadmap futuro).
- **Sincronización cloud:** no se contempla almacenamiento en la nube inicialmente.
- **Importación automática/API:** no hay conexión directa con servidores de Ges Deportivo (solo archivos Excel exportados manualmente por el usuario).
- Adicionalmente: análisis avanzado con IA, integración con sensores, y "no incorporar características demasiado complejas desde el inicio" — se prioriza una primera versión mínima y funcional sobre un sistema definitivo hecho de una sola vez.

1.4 Métricas de éxito

- **MTTR (Time to Report):** tiempo desde la carga del archivo hasta el informe completo < **2 minutos**.
- **Tasa de automatización:** % de estadísticas de partido generadas sin intervención manual > **95%**.
- **Disponibilidad offline:** **100%** de las funcionalidades críticas operativas sin conexión.
- **North Star Metric:** número de sesiones de análisis táctico realizadas por el DT por semana.
- **Tiempo de carga de datos:** la mayoría de los DTs encuestados solo están dispuestos a dedicar entre "menos de 10 minutos" y "10-20 minutos" por partido — es el techo operativo real.
- **Adopción por usabilidad:** los DTs prefieren explícitamente una app "simple y rápida" por sobre una "completa pero más compleja" — la simplicidad es un requisito de producto, no un nice-to-have.

2. Perfiles de Usuario

Perfil	Rol y Contexto	Objetivo Principal	Problema Actual
Director Técnico	Líder táctico, desde categorías formativas hasta profesional	Optimizar rendimiento mediante datos objetivos	El cálculo manual de EFF/PPP es tedioso

Perfil	Rol y Contexto	Objetivo Principal	Problema Actual
Analista / Asistente	Responsable de carga y procesamiento de datos	Proveer informes rápidos y precisos al DT	Carga redundante de datos ya existentes en Ges Deportivo

Perfil ampliado del usuario principal (Entrenador / Analista Estadístico)

- **Categorías que dirige:** todo el espectro — formativas (Mini, U13, U15, U17), de desarrollo (U21, Reserva), Primera Amateur, Veteranos y básquet Profesional.
- **Tamaño del cuerpo técnico:** variable — solos, en duplas, o equipos de 3+ personas.
- **Momento de análisis:** algunos toman datos durante el partido o en el club, pero la mayoría analiza las estadísticas **de forma diferida en sus casas**, horas o días después del partido.
- **Herramientas actuales:** papel y lápiz o Excel básico; algunos usan apps específicas.
- Satisfacción con los métodos actuales: media a baja.
- **Pain points textuales:**
 - *"pierdo el foco en la mejora"* del equipo durante el partido por tener que anotar/calcular.
 - El juego es *"demasiado frenético"* — demoras y pérdida de datos entre una carga y la siguiente.
 - *"tener que pasar el boxscore manualmente y calcular las estadísticas avanzadas por mi cuenta"* — frustración explícita con el cálculo manual.
 - No pueden *"identificar tendencias de hacia dónde va el básquet, ni identificar numéricamente errores del equipo"* — faltan datos como porcentajes reales por zona de cancha o situaciones puntuales (ej. goles rivales desde el 1v1).
 - *"no hay un espacio virtual donde se puedan dejar constancia de las estadísticas y que sea cómodo"* para medir rendimiento a corto/mediano/largo plazo.

3. Arquitectura de Datos

El sistema usa una base de datos relacional local (**SQLite**) con esquema normalizado.

Entidades principales

- **Gestión de identidad:** **usuario**, **club**, **usuarioClub** (N:M).
- **Estructura deportiva:** **jugador**, **categoría**, **competencia**, **inscripcion**.
- **Gestión de listas:** **listaBuenaFe**, **jugadorListaBuenaFe**.
- **Eventos y estadísticas:** **partido**, **jugadorPartido** (20+ métricas: puntos, T1/T2/T3, rebotes, asistencias, etc.).

Reglas de integridad

- Relación **1:1** entre **inscripcion** y **listaBuenaFe**.
- El historial de afiliaciones de un jugador se mantiene vía **jugadorClub** (N:M con fechas).
- **jugadorPartido** actúa como **fact table** para el motor de análisis (Pandas lee desde acá vía las vistas SQL).

4. Arquitectura de Software (Clean + Hexagonal)

Capas y responsabilidades

- **Dominio** (`src/dominio/`): entidades puras, reglas de negocio, excepciones e interfaces (ports), sin dependencias externas.
- **Aplicación** (`src/aplicacion/`, a crear): casos de uso y DTOs; orquesta reglas mediante inyección de dependencias.
- **Infraestructura** (`src/infraestructura/`): repositorios SQLite, parser Excel (Pandas), UI CLI/GUI y generación de reportes.

Regla de dependencias

```
dominio ← aplicación ← infraestructura
```

`dominio` no puede importar `aplicación` ni `infraestructura`. `aplicación` puede importar `dominio`, pero no `infraestructura`.

Patrones de diseño

- **Repository Pattern**: abstracción de persistencia por agregado (`UsuarioRepositorio`, `JugadorRepositorio`, etc.).
- **Dependency Injection**: los casos de uso reciben puertos (interfaces), no implementaciones concretas, por constructor.
- **Command Pattern**: CLI extensible por subcomandos sin bloques monolíticos `if/else`.

Estructura de directorios de referencia

```
src/
├── main.py
├── dominio/
│   ├── entidades/           # @dataclass puras, sin imports externos
│   ├── repositorios/       # interfaces (ABC) de repositorios
│   ├── exceptions.py       # excepciones de negocio
│   └── services/           # lógica de dominio compleja (opcional)
├── aplicacion/
│   ├── use_cases/          # orquestadores (reciben repos por DI)
│   ├── dtos/               # dataclasses de entrada/salida entre capas
│   └── services/           # servicios de aplicación (ej. SessionManager)
├── infraestructura/
│   └── repositorios/       # implementaciones SQLite de las interfaces de
dominio
│   ├── persistencia/
│   │   └── sql/            # schema.sql, views.sql, seed.sql, limpieza.sql
│   ├── analytics/         # motor Pandas
│   ├── ingest/            # parser Excel
│   ├── reports/           # generador PDF
│   ├── security/          # PasswordHasher
│   ├── ui/
│   └── cli/                # interfaz de línea de comandos
```

```

├── flet/                                # GUI
├── tests/ (hoy: test/)
│   ├── unit/ # sin DB, con mocks
│   ├── integration/ # con DB en memoria – hoy los tests viven flat en test/
│   └── conftest.py

```

¿Qué va en cada capa? Guía práctica

Capa de Dominio (`src/dominio/`) — el núcleo del sistema. No depende de ninguna tecnología concreta.

- **entidades/** — clases Python puras (`@dataclass`) que modelan los conceptos del básquet. Pueden tener métodos con lógica de negocio simple (validaciones, cálculos derivados). **No importan** `sqlite3`, `pandas` ni ningún framework. Ejemplo objetivo: `Jugador` con `calcular_edad()`; `EstadisticaJugador` con `validar_consistencia_puntos()`.
- **repositorios/** — interfaces abstractas (`ABC` + `@abstractmethod` en este proyecto — ver nota sobre `Protocol` en sección 19) que definen *qué* operaciones existen sobre los datos, sin decir *cómo*. Ejemplo real: `JugadorRepositorio` declara `buscar_por_id`, `buscar_por_dni`, `guardar`, sin una sola línea de SQL.
- **exceptions.py** — errores propios del negocio deportivo: `DNIDuplicadoError`, `JugadorNoHabilitadoError`, `PartidoInvalidoError` ("El club local no puede ser el visitante").

Capa de Aplicación (`src/aplicacion/`) — orquesta el dominio para los casos de uso del usuario. No contiene lógica de negocio pura (eso va en dominio) ni detalles técnicos (eso va en infraestructura).

- **use_cases/** — un archivo = una acción del usuario. Cada caso de uso recibe sus dependencias (repositorios, servicios) por constructor y expone un único método `execute(dto)`. Ejemplo: `RegistrarJugadorUseCase.execute(dto)` verifica DNI no duplicado (`player_repo.find_by_dni`), crea la entidad `Jugador`, la persiste (`player_repo.save`) y retorna un `JugadorDTO`.
- **dtos/**
- **services/** — servicios transversales que no pertenecen a un caso de uso específico, como `SessionManager` (sesión persistente) y `ExecutionContext` (propagación de `correlation_id` para logs).

Capa de Infraestructura (`src/infraestructura/`) — implementa los contratos del dominio con tecnologías concretas. Es la única capa que puede importar `sqlite3`, `pandas`, `flet`, `bcrypt`, etc.

- **repositorios/** — implementaciones SQLite de las interfaces de dominio. Cada clase hereda de su interfaz correspondiente (`SqliteJugadorRepositorio(JugadorRepositorio)`).
- **persistencia/** — infraestructura técnica de la base: `database_manager.py` (conexión, inicialización), archivos `.sql` (schema, vistas, seed), migraciones futuras. Sin lógica de negocio.
- **analytics/** — `formulas.py` (funciones puras sobre DataFrames), `pandas_analytics_service.py`, `chart_generator.py`.
- **ingest/** parser Excel de Ges Deportivo y servicio de ingesta.
- **reports/** generadores de PDF y reportería de CLI (leaderboards).
- **ui/** — subárbol separado por interfaz: `ui/cli/` y `ui/flet/`. Llamen a casos de uso; nunca acceden a la base de datos directamente.
- **security/** — hashing de contraseñas, política de credenciales, adaptador de cifrado de DB.

Data Transfer Objects (DTOs)

Un **DTO** es una clase simple cuya única responsabilidad es transportar datos entre capas — **no contiene lógica de negocio**.

¿Por qué se usan?

- **Desacoplamiento:** la UI no necesita conocer las entidades de dominio, y el dominio no se expone directamente al exterior.
- **Control de la frontera:** el DTO de entrada valida el formato antes de que el caso de uso lo procese; el DTO de salida define exactamente qué información se devuelve.
- **Evolución independiente:** se puede cambiar la entidad de dominio sin romper la UI, y viceversa.

Patrón de uso: cada caso de uso recibe un DTO de entrada (datos crudos del usuario) y retorna un DTO de salida (datos procesados para mostrar). Las entidades de dominio **nunca salen** de la capa de aplicación hacia la UI.

Ejemplo — registrar un jugador:

- `RegistrarJugadorInputDTO`: `nombre`, `apellido`, `dni`, `fecha_nacimiento`, `club_id`. Solo datos planos, sin métodos.
- El caso de uso recibe este DTO, crea la entidad `Jugador` con las reglas de dominio, y persiste.
- Retorna `JugadorOutputDTO`: `id`, `nombre_completo`, `dni`, `club_nombre`. Solo lo que la CLI/GUI necesita mostrar.

Ubicación: `src/aplicacion/dtos/`. Convención de nombres: `jugador_dto.py` puede contener tanto el DTO de entrada como el de salida para esa entidad.

Gestión de sesión y club activo

La sesión local persistirá en un archivo JSON en `~/.statspro/session.json` (o `./data/session.json` en desarrollo). El `SessionManager` es el único responsable de leer y escribir este archivo.

Estructura del archivo de sesión:

- `usuario_id`: identificador del usuario autenticado.
- `email`: email del usuario (para mostrar en la UI).
- `club_activo_id`: ID del club seleccionado actualmente (`null` si no se seleccionó ninguno).
- `session_token_hash`: hash del token de sesión (nunca el token en texto plano).
- `expires_at`: timestamp ISO 8601 de expiración.

¿Cómo se establece el `club_activo_id`?

1. Al hacer login, el `SessionManager` crea la sesión con `club_activo_id = null`.
2. El usuario ejecuta `stats club select <id>`.
3. El comando llama a `CambiarClubActivoUseCase`, que verifica que el club pertenezca al usuario.
4. Si la verificación pasa, el `SessionManager` actualiza `club_activo_id` en el archivo.
5. Los comandos operativos (cargar partido, listar jugadores) leen `club_activo_id` al inicio; si es `null`, abortan con un mensaje claro.

¿Por qué club activo y no pasarlo como argumento? Un DT trabaja siempre en el contexto de un club. Tener el club activo en la sesión evita escribir `--club-id 3` en cada comando — el mismo patrón que usan los IDEs con el "proyecto activo" o las shells con el "directorio actual".

Composition Root: cómo se ensambla la aplicación

El **Composition Root** es el único lugar del sistema donde se instancian todas las dependencias y se conectan entre sí. En esta arquitectura, ese lugar será `src/infraestructura/ui/cli/main_cli.py` (CLI) y `src/infraestructura/ui/flet/app.py` (GUI, Hito 4).

¿Por qué es importante? Porque en todos los demás archivos, las clases reciben sus dependencias como argumentos (nunca las crean con instanciación directa). Esto hace el sistema testeable: en los tests se pueden pasar repositorios falsos (mocks) sin modificar el código de producción.

Flujo de ensamblaje esperado en `main_cli.py` (crece con cada US):

1. **US-101/102:** se instancia `SQLiteManager`, se ejecutan las migraciones/inicialización, se crean los repositorios SQLite pasando la conexión.
2. **US-103/104:** se crean los servicios de aplicación (`SessionManager`) y los casos de uso administrativos, pasando los repositorios.
3. **US-105/106:** se crean los casos de uso operativos y se registran todos los subcomandos CLI.
4. **US-107:** se inicializa el `ExecutionContext` antes de despachar cualquier comando y se conecta el logging.
5. **US-401 (GUI):** mismo patrón en `app.py`, pero las pantallas Flet reciben los casos de uso como dependencias.

Regla de oro: si una clase crea sus dependencias con `NombreClase()` dentro de un método que no sea el composition root, hay un problema de acoplamiento que debe corregirse.

5. Acuerdo de Ingeniería y Estándares

Principios de desarrollo

- **Diseño precede a la implementación:** no se escribe código sin un diseño previo aprobado (ADR).
- **Atomicidad:** commits pequeños y lógicos. Formato: `tipo(alcance): descripción`.
- **Gestión de ramas:** `feature/nombre-tarea`, `hotfix/descripción`, `release/vX.Y`.
- **Higiene del repositorio:** prohibido subir binarios, bases SQLite con datos reales, o archivos temporales.

Calidad de código y pruebas

- **Documentación:** estilo Javadoc/Doxygen (docstrings) para módulos y métodos públicos.
- **Pruebas:** cobertura mínima 80% en lógica de negocio; 95% en componentes críticos (ver Catálogo de Criticidad, sección 14).
- **Análisis estático:** `ruff`.

Gestión de tareas — prioridad y esfuerzo

Prioridad	Descripción
Urgente	Bloqueante; detiene el desarrollo de otras US
Alta	Impacto directo en la entrega del hito
Media	Importante pero no bloquea el progreso
Baja	Mejora o refinamiento posterior
Tamaño	Esfuerzo estimado
XS	Menos de 1 día hábil
S	1–2 días hábiles
M	3–5 días hábiles
L	6–10 días hábiles

6. Reglas de Negocio Consolidadas

Centraliza las reglas obligatorias del dominio para que desarrollo y testing sean coherentes en todas las historias de usuario.

Identidad y Seguridad:

- Email de usuario único por sistema.
- Contraseña nunca persistida en texto plano ni en logs.
- Sesión requiere usuario autenticado y, para comandos operativos, club activo.

Jugadores, Clubes y Afiliaciones:

- DNI de jugador único cuando está informado.
- Un jugador no puede tener dos vínculos activos superpuestos con el mismo club.
- Historial de afiliación coherente en fechas: `fecha_hasta >= fecha_desde`.

Competencias, Inscripciones y Listas:

- Una inscripción es única por club + competencia + categoría + temporada.
- Cada inscripción tiene una única lista de buena fe asociada (1:1).
- Solo jugadores habilitados en lista pueden figurar en carga oficial de partido.

Partidos y Estadísticas:

- Un partido no puede tener el mismo club como local y visitante.
- Toda carga de partido y boxscore es atómica (todo o nada).
- Estadísticas de tiro: convertidos $\leq$ lanzados; todos los valores no negativos.
- Puntos de jugador = $T1C + T2C \times 2 + T3C \times 3$ (coherencia verificada).
- Minutos por jugador no pueden exceder el máximo reglamentario definido por competencia.

Analítica y Reporting:

- Fórmulas avanzadas deben manejar división por cero y no devolver `NaN/inf`.

- Reportes y dashboards se construyen sobre vistas SQL normalizadas y versionadas.
- Toda exportación (PDF/backup) debe ser trazable en logs con timestamp y resultado.

7. Requisitos No Funcionales (NFR)

ID	Requisito	Medición / Umbral	Severidad
NFR-1	Portabilidad	Ejecución nativa en Windows 10+, Android 9+, iOS 14+	Bloqueante
NFR-2	Rendimiento Ingesta	Procesamiento de Excel con Pandas < 5 seg	Alta
NFR-3	Fiabilidad de Datos	Integridad referencial en SQLite (FKs activas)	Bloqueante
NFR-4	Usabilidad	Carga de partido completo en < 3 clics desde selección de archivo	Media
NFR-5	Arranque	Tiempo de inicio de la GUI < 3 seg en entorno objetivo	Alta
NFR-6	Offline	100% de funcionalidades críticas sin conexión a internet	Bloqueante
NFR-7	Cobertura de Tests	≥80% no críticos; ≥95% en módulos críticos (ver Catálogo de Criticidad)	Alta

8. Registro de Decisiones Arquitectónicas (ADR)

ID	Título	Estado	Decisión	Bloquea
ADR-001	Arquitectura Local-First	 Aprobado	SQLite + offline-first	Hito 1
ADR-002	Framework UI	 Pendiente	Flet (Python puro) vs. Compose Multiplatform	Hito 4
ADR-003	Protocolo de Ingesta Excel	 Pendiente	Estandarizar mapeo/limpieza de columnas de Ges Deportivo	US-201
ADR-004	Versionado de DB	 Pendiente	Migraciones manuales (<code>schema_version</code>) vs. Alembic	Hito 2
ADR-005	Reportes PDF	 Pendiente	<code>reportlab</code> (sin dependencias externas) vs. <code>weasyprint</code> (HTML→PDF)	US-302
ADR-006	Seguridad y Cifrado	 Pendiente	Hash de passwords y eventual cifrado DB (SQLCipher)	US-106 / Hito 4
ADR-007	Motor de Visualización	 Pendiente	<code>matplotlib</code> (offline) vs. <code>plotly</code> (interactivo, requiere servidor local)	US-301

ID	Título	Estado	Decisión	Bloquea
ADR-008	Estrategia de Backup	 Pendiente	Exportación/restauración de base local y versiones	Hito 4
ADR-009	Pipeline CI/CD	 Pendiente	GitHub Actions para lint, tests y cobertura automáticos	US-108

Estructura canónica de un ADR (ver [docs/documentacion_app_estadistica/ADR/template_adr.md](#)): Contexto → Decisión → Alternativas Consideradas → Consecuencia (Positivas / Negativas / Restricciones). Ninguno de los 9 está escrito todavía.

9. Hito 1 — Núcleo de Datos e Interfaz CLI (v0.1)

Objetivo del hito: construir una base técnica funcional por CLI con persistencia robusta, autenticación local, casos de uso operativos, validaciones de integridad y un entorno de calidad que garantice reproducibilidad desde el primer commit. Sistema funcional por línea de comandos con persistencia robusta.

Épicas: 3 · **Historias:** 8 · **Esfuerzo total estimado:** ~50 días·persona

Épica H1-E1: Infraestructura y Persistencia

US-101 — Esquema SQLite, Vistas y Datos Semilla

- **Esfuerzo:** L (6-10 días) · **Prioridad:** Urgente (bloqueante)
- **Objetivo Funcional:** habilitar un esquema relacional local verificable y un conjunto de vistas operativas que sirvan de contrato estable para todo el ciclo del producto, garantizando que Pandas y la capa de aplicación consuman datos sin transformaciones ambiguas.
- **Narrativa:** Como desarrollador, quiero el esquema relacional completo en SQLite, con sus vistas de análisis y datos de prueba, para tener una base verificable sobre la que construir el sistema.
- **Capa de Infraestructura:**
 - **Clase `SQLiteManager`** (`src/infraestructura/persistencia/database_manager.py`):
 - `connect()` / `conectar()`: retorna una conexión activa con `PRAGMA foreign_keys = ON` y `row_factory = sqlite3.Row`.
 - `inicializar_schema()` / `inicializar_db()`: ejecuta de forma atómica los scripts `schema.sql`, `views.sql` (y opcionalmente `seed.sql`) usando `executescript()`.
 - **Scripts SQL** (`src/infraestructura/persistencia/sql/`):
 - `schema.sql`: DDL completo — tablas con tipos estrictos, PK, FK, `CHECK` constraints, `CREATE TABLE IF NOT EXISTS` y `DROP TABLE IF EXISTS` en orden inverso de dependencias.
 - `views.sql`: las 4 vistas de análisis estadístico.
 - `seed.sql`: datos de prueba (1 usuario, 2 clubes, 10 jugadores, 1 competencia, ≥ 2 partidos con boxscore).
- **Vistas a implementar:**
 1. `v_partidos_resumen`: un partido con clubes y competencia (reemplaza IDs por nombres).
 2. `v_boxscore_completo`: un `jugadorPartido` con jugador y club (fuente principal para Pandas).
 3. `v_jugador_totales_temporada`: acumulados históricos por jugador y año de competencia.
 4. `v_listas_detalle`: jugadores habilitados por inscripción.

- **Criterios de Aceptación:**

- **AC1.** Schema idempotente: `CREATE TABLE IF NOT EXISTS` en toda la DDL, con `DROP TABLE IF EXISTS` en orden inverso de dependencias.
- **AC2.** FKs activas con reglas `ON DELETE/UPDATE CASCADE` en relaciones críticas.
- **AC3.** `CHECK` constraints en métricas numéricas (ej. `puntos >= 0`, `minutosJugados BETWEEN 0 AND 48`) y en integridad lógica (ej. `idClubLocal != idClubVisitante`).
- **AC4.** El campo `dni` en `jugador` es `UNIQUE` pero permite `NULL`.
- **AC5.** Las 4 vistas exponen columnas con nombres y tipos estables, documentados; todas las divisiones usan `NULLIF/CASE` para nunca fallar por división por cero.
- **AC6.** `seed.sql` se ejecuta limpiamente sobre un schema vacío y puebla todas las tablas con datos significativos para las vistas.
- **AC7.** Columnas de vistas estables para consumo desde Pandas sin transformaciones adicionales.

- **Reglas de Negocio (nivel DB):**

- `CHECK(idClubLocal != idClubVisitante)` en `partido`.
- `fechaHasta >= fechaDesde` en historial de afiliaciones.
- `UNIQUE` en `listaBuenaFe.idInscripcion` (refuerza la relación 1:1).

- **Entidades/Modelos implicados (tablas):** `usuario`, `club`, `usuarioClub`, `jugador`, `categoría`, `competencia`, `inscripcion`, `listaBuenaFe`, `jugadorListaBuenaFe`, `jugadorClub`, `partido`, `jugadorPartido`.

- **Testing Mínimo:**

- *Integración* (`test_database.py`): `test_database_schema` (existencia de tablas/vistas), `test_referential_integrity` (FK inexistente → `IntegrityError`), `test_check_constraints` (valores negativos → falla), `test_seed_execution` (vistas devuelven datos tras el seed), `test_division_by_zero` (vistas devuelven 0.0, nunca error).

US-102 — DatabaseManager y Patrón Repository

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Urgente (bloqueante) · **Dependencias:** US-101
- **Objetivo Funcional:** implementar el orquestador de conexión y las interfaces de persistencia bajo Clean Architecture, asegurando que el acceso a datos sea independiente del motor de base de datos y garantizando la integridad referencial.
- **Narrativa:** Como desarrollador, quiero una capa de infraestructura que gestione el ciclo de vida de la conexión SQLite y exponga repositorios tipados para cada agregado del dominio.
- **Capa de Dominio — Interfaces (`src/dominio/repositorios/`):** las firmas reales ya migraron de "parámetros sueltos" a **recibir la dataclass completa**
 - `UsuarioRepositorio` (`UserRepository`): `encontrar_por_mail(email)/get_by_email`, `encontrar_por_id(id)/get_by_id`, `guardar(usuario: Usuario)/save`.
 - `ClubRepositorio` (`ClubRepository`): `buscar_por_id_usuario/get_clubs_by_user`, `buscar_por_id/get_by_id`, `buscar_por_nombre`, `guardar(club: Club)/save`, `link_user_to_club(us_club: UsuarioClub)`.
 - `JugadorRepositorio` (`PlayerRepository`): `buscar_por_id`, `buscar_por_dni/search_by_dni`, `buscar_por_club`, `guardar(jugador: Jugador)`, `link_to_club(jc: JugadorClub)`, `club_activo/get_active_club`.
 - `CompetenciaRepositorio` (`CompetitionRepository`): `guardar_competencia(compe: Competencia)/save_competencia`, `buscar_competencia_por_id/get_competencia_by_id`, `obtener_todas_competencias/get_all_competencias`,

```

guardar_categoria(cat:Categoria)/save_categoria,
obtener_categorias/get_categorias, guardar_inscripcion(inscripcion:
Inscripcion)/save_inscripcion, buscar_inscripcion_por_id/get_inscripcion_by_id,
obtener_inscripciones_por_club/get_inscripciones_by_club,
guardar_lista_buena_fe(listaBF: ListaBuenaFe)/save_lista_buena_fe,
obtener_lista_por_inscripcion/get_lista_by_inscripcion (tipada -> ListaBuenaFe
singular, coherente con la relación 1:1), agregar_jugador_lista(idJugador,
idListaBuenaFe)/add_jugador_to_lista y
obtener_jugadores_lista/get_jugadores_by_lista.
◦ JuegoRepositorio (GameRepository): buscar_por_club, buscar_por_id,
guardar_partido(partido: Partido)/save_partido,
guardar_boxscore(boxscore: JugadorPartido)/save_boxscore

```

- **Capa de Infraestructura:**

- **Clase SQLiteManager:** administra la conexión (`sqlite3.Connection`); `connect()` activa `PRAGMA foreign_keys` y `row_factory = sqlite3.Row`, retornando la conexión activa si ya existe; `initialize_schema()/inicializar_schema()` ejecuta `schema.sql + views.sql` en una sola llamada atómica.
- **Implementaciones concretas** (`src/infraestructura/repositorios/`):
`SqliteUsuarioRepositorio`, `SqliteClubRepositorio`, `SqliteJugadorRepositorio`,
`SqliteCompetenciaRepositorio`, `SqliteJuegoRepositorio`.
`SqliteCompetenciaRepositorio` maneja `competencia`, `categoria`, `inscripcion`,
`listaBuenaFe` y `jugadorListaBuenaFe` como un único agregado competitivo. Cada repositorio mapea manualmente `sqlite3.Row` a las dataclasses de dominio mediante un método privado `_row_to_entity()`.

- **Criterios de Aceptación:**

- **AC1 — Gestión de Conexión:** `connect()` garantiza integridad referencial y acceso por nombre de columna.
- **AC2 — Abstracción Total:** la capa `dominio/` no importa `sqlite3`, `pandas` ni ninguna librería de infraestructura.
- **AC3 — Mapeo de Datos:** los repositorios retornan objetos `@dataclass` puros, nunca tuplas de SQLite.
- **AC4 — Transaccionalidad:** `SqliteJuegoRepositorio.guardar_boxscore()` (y, a futuro, un método combinado tipo `save_with_boxscore`) debe permitir transacciones multi-tabla para asegurar la integridad de la carga de partidos. **Hoy no se cumple, y el problema es previo a la transaccionalidad:** `SqliteJuegoRepositorio` ni siquiera se puede ejecutar tal como está — `guardar_partido` inserta contra una tabla `Juego` que no existe (la real es `partido`) y llama a `self.conexion.obtener_conexion()`, método que no existe sobre un `sqlite3.Connection` crudo (`AttributeError` garantizado); `guardar_boxscore` tiene la lista de columnas del `INSERT` con 19 nombres (falta `idClub`) pero 20 valores en la tupla (`sqlite3.ProgrammingError` garantizado). Antes de pensar en envolver esto en una transacción multi-tabla, hay que corregir estos tres bugs.

- **Reglas de Negocio:**

- Validación de DNI duplicado al guardar un jugador (lanza excepción de dominio).
- Uso de `cursor.lastrowid` para retornar la entidad con el ID asignado por la base de datos.

- **Entidades/Modelos implicados:** `Usuario`, `Club`, `Jugador`, `JugadorClub`, `Competencia`, `Categoria`, `Inscripcion`, `ListaBuenaFe`, `JugadorListaBuenaFe`, `Partido`, `EstadisticaJugadorPartido`.

- **Vistas SQL necesarias:** `v_partidos_resumen` y `v_boxscore_completo` para optimizar las consultas de lectura en los repositorios.
- **Testing Mínimo:**
 - *Integración* (`tests/integration/test_repositories.py`): CRUD completo por repositorio usando DB `:memory::`; `buscar_por_dni` retorna `None` si no existe, sin lanzar excepción; el `save()` de un partido y sus estadísticas es atómico; los repositorios de lectura usan las vistas SQL correctamente.

Archivos a crear (consolidado, nombres en español):

src/dominio/repositorios/	
— usuario_repositorio.py	✓ existe
— club_repositorio.py	✓ existe
— jugador_repositorio.py	✓ existe
— competencia_repositorio.py	✓ existe
— juego_repositorio.py	✓ existe
src/infraestructura/repositorios/	
— sqlite_usuario_repositorio.py	✓ existe, funcional y testeado
— sqlite_club_repositorio.py	✓ existe, funcional (sin tests dedicados todavía)
— sqlite_jugador_repositorio.py	⚠ existe, conexión OK, pero <code>`link_to_club`</code> usa atributos
	que no existen en <code>`JugadorClub`</code>
	(<code>`id_jugador`/`id_club`</code> en vez de <code>`idJugador`/`idClub`</code>) – ver sección 20
— sqlite_competencia_repositorio.py	⚠ EXISTE – pero con 3 bugs y 2 métodos sin implementar (<code>`pass`</code>) – ver sección 20
— sqlite_juego_repositorio.py	⚠ existe, conexión OK, pero no es funcional: tabla/columna equivocada, método de conexión inexistente, <code>`INSERT`</code> desalineado – ver sección 20

Estado real: las 5 implementaciones ya existen y las 5 usan el mismo patrón de conexión (`sqlite3.Connection` crudo por constructor). Lo que falta es corregir bugs puntuales de tres de ellas.

Épica H1-E2: Lógica de Aplicación y CLI

US-103 — Gestión de Entidades (Casos de Uso Administrativos)

- **Esfuerzo:** L (6-10 días) · **Prioridad:** Alta · **Dependencias:** US-101, US-102
- **Objetivo Funcional:** implementar la lógica de negocio pura y la interfaz de usuario por comandos para la gestión integral de las entidades del sistema (jugadores, clubes, competencias, inscripciones), asegurando la validación de reglas deportivas y la integridad de los datos.
- **Narrativa:** Como administrador, quiero disponer de casos de uso con lógica de negocio validada para gestionar el ciclo de vida de los jugadores y sus afiliaciones, así como la estructura de competencias y clubes.

- **Capa de Dominio:**
 - **Entidades:** `Usuario`, `Club`, `Jugador`, `JugadorClub` (historial N:M jugador-club), `Competencia`, `Categoria`, `Inscripcion`, `ListaBuenaFe`, `JugadorListaBuenaFe`, `Partido`, `EstadisticaJugador`. `@dataclass` puras, serializables, sin dependencias externas.
 - **Lógica de validación:** en `__post_init__` de las entidades (ej. tiros convertidos $\leq$ lanzados, valores no negativos) — **pendiente, hoy las entidades son dataclasses simples sin validación** (ver sección 20).
 - **Excepciones** (`src/dominio/exceptions.py`, a crear): `JugadorDuplicadoError`, `ClubNoEncontradoError`, `UsuarioNoEncontradoError`, `CredencialesInvalidasError`, `VinculoActivoExistenteError`.
- **Capa de Aplicación (a crear):**
 - **Casos de uso:** `RegistrarJugadorUseCase` (valida DNI numérico, no vacío y no duplicado), `CrearClubUseCase`, `VincularJugadorAClubUseCase` (evita vínculos activos duplicados), `CrearCompetenciaUseCase`, `InscribirClubEnCompetenciaUseCase` (genera automáticamente la `listaBuenaFe` vacía asociada, 1:1 — **este caso de uso depende de que primero se corrija `SqliteCompetenciaRepositorio.obtener_lista_por_inscripcion`**: hoy la implementación devuelve `list[ListBuenaFe]` pese a que tanto la interfaz de dominio como la relación 1:1 real dicen que debería devolver `ListaBuenaFe | None`; si se construye el caso de uso contra el comportamiento actual, va a tratar como "lista de listas de buena fe" algo que conceptualmente es un objeto único — ver sección 20), `ListarClubesUsuarioUseCase`, `ListarJugadoresClubUseCase`, `ListarPartidosPorClubUseCase` (usa `v_partidos_resumen`), `CambiarClubActivoUseCase`.
 - **DTOs:** `JugadorDTO`, `ClubDTO`, `CompetenciaDTO`, `CrearJugadorDTO`, `PartidoResumenDTO`, `InscripcionDTO`.
- **Capa de Infraestructura:**
 - **Comandos CLI** (`src/infraestructura/ui/cli/commands/`, a crear): `player_add.py`, `club_add.py`, `player_link.py`, `game_list.py`, `player_list.py`, `club_list.py`.
 - Command Pattern con `argparse`; prompts interactivos (`input()`); formateo con `tabulate`.
- **Criterios de Aceptación:**
 - **AC1 — Independencia de Dominio:** los archivos en `dominio/entidades/` no importan librerías externas.
 - **AC2 — Inyección de Dependencias:** todos los casos de uso reciben sus repositorios vía constructor, usando las interfaces (protocolos/ABC).
 - **AC3 — Validación Fail-Fast:** DNI duplicado o datos inválidos cortan el flujo de la CLI con mensajes de error amigables, sin tracebacks.
 - **AC4 — Formato de Salida:** la CLI siempre formatea resultados exitosos y listas con tablas en consola (`tabulate`).
 - **AC5 — Atomicidad:** operaciones complejas (inscripciones que crean listas de buena fe) son atómicas.
- **Reglas de Negocio:**
 - DNI de jugadores numérico y único.
 - Un jugador no puede estar vinculado activamente (sin `fecha_hasta`) a más de un club (ni al mismo club dos veces).
 - Porcentajes y totales en estadísticas se validan antes de la persistencia.
- **Testing Mínimo:**

- *Unitarios*: validar excepciones en `EstadisticaJugador` por datos incoherentes; mocks de repositorios para `RegistrarJugador` (DNI duplicado) y `VincularJugadorAClub`; propiedades calculadas (`nombre_completo`, `rebotes_totales`).
- *Integración*: persistencia real en DB `:memory:` y validación de consultas vía DTOs.

Archivos a crear:

```
src/dominio/entidades/
├─ usuario.py           ✓ existe (simple, sin validaciones aún)
├─ club.py              ✓ existe
├─ jugador.py           ✓ existe
├─ jugador_club.py      ✓ existe (dentro de jugador.py)
├─ competencia.py       ✓ existe
├─ categoria.py         ✓ existe (dentro de competencia.py)
├─ inscripcion.py        ✓ existe (dentro de competencia.py)
├─ lista_buena_fe.py     ✓ existe (dentro de competencia.py)
├─ jugador_lista_buena_fe.py ✓ existe (dentro de competencia.py)
├─ partido.py           ✓ existe
└─ estadistica_jugador.py ✓ existe (como JugadorPartido, dentro de
partido.py)

src/aplicacion/use_cases/ (✗ ninguno existe todavía – capa aplicación no
creada)
├─ registrar_jugador.py
├─ crear_club.py
├─ vincular_jugador_club.py
├─ crear_competencia.py
├─ inscribir_club_competencia.py
├─ listar_clubes_usuario.py
├─ listar_jugadores_club.py
├─ listar_partidos_por_club.py
└─ cambiar_club_activo.py
```

Nota de organización real: el PRD prevé un archivo por entidad/caso de uso; el código actual agrupa varias entidades relacionadas en un mismo archivo (ej. `competencia.py` contiene `Competencia`, `Categoria`, `Inscripcion`, `ListaBuenaFe` y `JugadorListaBuenaFe`). Es una decisión de organización razonable para el tamaño actual — no es un error, pero vale la pena un acuerdo explícito del equipo sobre si se mantiene así o se separa 1:1 como sugiere el PRD (ver sección 20).

US-104 — Autenticación y Sesión Local

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Alta · **Dependencias:** US-103
- **Objetivo Funcional:** permitir el registro y acceso seguro de entrenadores al sistema, manteniendo un estado de sesión persistente entre ejecuciones de la CLI para evitar solicitudes repetitivas de credenciales y el ingreso constante del ID del club activo.
- **Narrativa:** Como usuario, quiero un sistema de login local que proteja mis datos y mantenga mi sesión entre ejecuciones de la CLI.
- **Capa de Dominio:**
 - **Entidades:** `Usuario` (`id`, `nombre`, `email`, `password_hash`, `salt`).

- **Excepciones:** `EmailYaRegistradoError`, `UsuarioNoEncontradoError`, `CredencialesInvalidasError`.
- **Interfaces:** `UsuarioRepositorio` (`get_by_email`, `get_by_id`, `save`).
- **Capa de Aplicación:**
 - **Casos de uso:** `RegistrarEntrenadorUseCase`, `LoginLocalUseCase`.
 - **DTOs:** `RegistrarDTO/RegisterInputDTO`, `LoginDTO/LoginInputDTO`, `SessionDTO` (`usuario_id`, `email`, `club_activo_id`).
 - **Servicios de Aplicación:** `SessionManager` (`load_session/get_current`, `save_session`, `is_authenticated`, `clear_session/destroy`, `set_active_club/set_club_activo`).
- **Capa de Infraestructura:**
 - **Seguridad:** `PasswordHasher` — wrapper sobre `bcrypt/argon2-cffi` (objetivo final) con solo dos métodos públicos: `hash(password)` y `verify(password, hash)`. (v0.1 puede arrancar con `hashLib.pbkdf2_hmac/SHA-256` con salt dinámico según ADR-006, migrando a `bcrypt/Argon2` después — ver tabla de ADRs.)
 - **Persistencia:** `SqliteUsuarioRepositorio`.
 - **Gestión de sesión:** `SessionManager` persiste `usuario_id` y `club_activo_id` en un JSON oculto (`~/.statspro/session.json` o `~/.statspro_session.json`).
 - **CLI:** `stats auth register`, `stats auth login`, `stats auth logout`, `stats club select <id>`.
- **Base de Datos:** tabla `usuario` (`idUsuario`, `nombre`, `email`, `contrasenia` — nombre real de columna, ver nota sobre el campo `pw` en sección 20).
- **Criterios de Aceptación:**
 - **AC1 — Seguridad de Credenciales:** las contraseñas NUNCA se almacenan ni se loguean en texto plano. Hashing determinista con salt.
 - **AC2 — Persistencia de Sesión:** la sesión sobrevive al cierre de la CLI; al reiniciar, `is_authenticated()` retorna `True` si había sesión activa.
 - **AC3 — Manejo de Contexto:** el archivo de sesión recuerda el club activo actual.
 - **AC4 — Validaciones:** email único; contraseña con requisitos mínimos (≥ 6 caracteres en v0.1; ≥ 12 con complejidad en v1.0, ver US-403).
- **Reglas de Negocio:**
 - `clear_session()` es idempotente.
 - `set_active_club()` falla si no hay sesión previa.
 - Los comandos protegidos ejecutan `require_auth()` y `require_active_club()` según corresponda.
- **Testing Mínimo:**
 - *Unitarias:* hash y verificación de contraseñas; lógica de registro/login con repositorios mock.
 - *Integración:* persistencia de sesión con archivo temporal; flujo completo registro → login → sesión usando DB `:memory:`.

Archivos a crear:

```
src/infraestructura/security/
└─ password_hasher.py

src/aplicacion/
├─ services/session_manager.py
└─ use_cases/
```

```

├─ registrar_entrenador.py
├─ login_local.py

src/aplicacion/dtos/
├─ auth_dto.py

test/
├─ test_auth.py

```

US-105 — Carga Atómica de Partido (CargarPartido)

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Alta · **Dependencias:** US-103, US-104
- **Objetivo Funcional:** registrar un evento de partido y sus estadísticas individuales asociadas garantizando la integridad de los datos mediante una transacción atómica (todo o nada).
- **Narrativa:** Como DT, quiero registrar un partido completo con todas las estadísticas de los jugadores en una única operación; si falla una sola estadística, nada se persiste.
- **Capa de Dominio:**
 - **Entidades:** **Partido** (id, competencia, fecha, estadio), **EstadisticaJugador** (idJugador, idPartido, puntos, tiros, rebotes, etc.).
 - **Interfaces:** **JuegoRepositorio** con método **save_partido_completo(partido, boxscore: list[EstadisticaJugador])** (a agregar — hoy el método existente es **guardar_boxscore** fila por fila, ver sección 20). **Bloqueo real, no solo un método por agregar:** **SqliteJuegoRepositorio.guardar_partido** y **.guardar_boxscore** hoy no son funcionales en absoluto (tabla **Juego** inexistente, llamada a un método de conexión que no existe, **INSERT** con columnas/valores desalineados — ver sección 20) — esta US no se puede construir sobre la implementación actual sin corregir esos tres bugs primero, además de sumar el método combinado atómico.
- **Capa de Aplicación:**
 - **Caso de uso:** **CargarPartidoUseCase** (orquestra validación y persistencia).
 - **DTOs:** **PartidoDTO**, **BoxscoreDTO**, **EstadisticaInputDTO**.
- **Capa de Infraestructura:**
 - **Persistencia:** **SqliteJuegoRepositorio.save_partido_completo** — debe usar un context manager de SQLite (**with self.connection:**) para envolver el **INSERT** de **partido** y los múltiples **INSERT** de **jugadorPartido** en una sola transacción BEGIN/COMMIT.
 - **CLI:** **stats game add** con flujo interactivo multi-paso.
- **Criterios de Aceptación:**
 - **AC1 — Atomicidad Garantizada:** si falla la inserción del boxscore en cualquier punto (ej. jugador 8 de 12), se ejecuta **ROLLBACK** y no se guarda el partido huérfano.
 - **AC2 — Validaciones Pre-persistencia:** la validación de DTOs y reglas de negocio ocurre ANTES de la primera operación de base de datos; el mensaje de error especifica qué jugador y qué campo causó el error.
 - **AC3 — Independencia:** el caso de uso no contiene SQL embebido — se delega totalmente al repositorio.
- **Reglas de Negocio:**
 - $(T1C \times 1) + (T2C \times 2) + (T3C \times 3) = \text{puntos totales}$.
 - $\text{convertidos} \leq \text{lanzados}$ para T1, T2, T3.
 - Todos los campos numéricos ≥ 0 .

- `minutosJugados` no excede el total del partido (ej. 48 min).
- No se puede cargar un partido si los clubes involucrados no existen en la DB.
- **Nota (campo propuesto, no implementado todavía):** si se agrega a `Partido` un resultado final (`puntosLocalFinal/puntosVisitanteFinal`, ver sección 20 y el DER en `docs/diagramas/diagramas.md`), esta US sería el lugar natural para completarlo — junto con una regla que valide que la suma de puntos del boxscore por club coincide con ese resultado final antes de persistir.
- **Testing Mínimo:**
 - *Unitarias:* fallo de persistencia total ante una sola estadística inválida; el mensaje de error especifica jugador y campo.
 - *Integración:* flujo completo (partido + boxscore) en DB `:memory:` con repositorios reales.

Archivos a crear:

```
src/aplicacion/
├── dtos/partido_dto.py
└── use_cases/cargar_partido.py

test/
├── test_use_case_cargar_partido.py
└── test_cargar_partido_atomicidad.py
```

US-106 — CLI con Command Pattern

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Alta · **Dependencias:** US-103, US-104, US-105
- **Narrativa:** Como administrador, quiero una CLI estructurada con subcomandos claros para gestionar todas las entidades, que muestre los datos en tablas formateadas.
- **Objetivo Funcional:** construir una CLI extensible y mantenible con subcomandos desacoplados que consolide todo el flujo operativo de v0.1, sin bloques monolíticos `if/else`.
- **Comandos (usar `argparse`):**

```
stats auth register / login / logout

stats club add           → solicita nombre por input()
stats club list          → tabla con v_partidos_resumen (filtrado por
usuario)
stats club select <id>   → actualiza sesión

stats player add         → formulario interactivo campo por campo
stats player list        → tabla: ID | Nombre | DNI | Club Activo
stats player link <id_jugador> → solicita id_club y fecha_desde

stats game add           → formulario multi-paso
stats game list          → tabla con v_partidos_resumen
stats game boxscore <id_partido> → tabla con v_boxscore_completo
```

- **Archivos a crear:**

```

src/infraestructura/ui/cli/
├─ main_cli.py           ← punto de entrada / composition root
├─ commands/
│   ├── __init__.py
│   ├── auth_commands.py ← register, login, logout
│   ├── club_commands.py  ← add, list, select
│   ├── player_commands.py ← add, list, link
│   └── game_commands.py  ← add (interactivo), list, boxscore
└─ formatters/
    └─ table_formatter.py ← TableFormatter (wrapper de tabulate)

```

- **Criterios de Aceptación:**

- **AC1 — Command Pattern:** agregar un nuevo grupo de comandos (ej. `stats competition ...`) solo requiere crear un nuevo archivo en `commands/` y registrarlo en `main_cli.py` — sin modificar ningún otro archivo. Las excepciones de dominio nunca muestran `tracebacks` al usuario final.
- **AC2 — Visualización con vistas SQL:** `stats game list` usa `v_partidos_resumen` (nombres de clubes, no IDs); `stats game boxscore <id>` usa `v_boxscore_completo`; `stats player list` muestra el club activo del jugador (del historial `jugadorClub`).
- **AC3 — Flujo de sesión:** los comandos `club`, `player` y `game` ejecutan `require_auth()` al inicio; `game` y `player list` ejecutan `require_active_club()`.

- **Testing Mínimo:**

- *Unitario:* flujo de guards de autenticación y club activo.
- *Integración:* ejecución de cada subcomando con DB en memoria, verificando salida esperada; comando sin sesión → mensaje de error controlado.

US-107 — Monitoreo y Trazabilidad Operativa

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Media · **Dependencias:** US-106
- **Objetivo Funcional:** proveer observabilidad transversal estructurada para depuración y soporte, con correlación de eventos extremo-a-extremo por ejecución de comando.
- **Archivos a crear:**

```

src/infraestructura/logging/
├─ logger_config.py      ← configuración de sinks (local-file / Seq)
└─ seq_handler.py        ← handler HTTP para Seq (opcional)

src/aplicacion/services/
└─ execution_context.py  ← genera correlation_id UUIDv4 y propaga contexto

docker-compose.yml       ← perfil "observabilidad" con Seq (opcional)

test/
├─ test_execution_context.py
└─ test_logging_redaction.py

```

- **Eventos de log obligatorios:** inicio/fin de comando CLI (resultado + duración); login, logout, cambio de club activo, fallos de autenticación; inicio/fin de transacciones críticas (carga partido, importación Excel, backup/restore); creación/actualización de entidades principales; excepciones de dominio y errores de infraestructura, clasificados por severidad.
- **Criterios de Aceptación:**
 - **AC1.** Niveles DEBUG/INFO/WARNING/ERROR en formato JSON estructurado.
 - **AC2.** `correlation_id` UUIDv4 generado al inicio de cada comando y propagado hasta repositorios.
 - **AC3.** Secretos (passwords, tokens, hashes sensibles) redactados en todos los niveles de log.
 - **AC4.** Selección de sink (local-file o Seq) por configuración, sin cambiar código de negocio.
 - **AC5.** En modo Seq, eventos filtrables por `correlation_id`, `command_name` y nivel.
- **Testing Mínimo:**
 - *Unitario:* creación y propagación de `execution_context`; redacción de secretos en campos conocidos.
 - *Integración:* flujo CLI completo → verificar que cada evento tiene el mismo `correlation_id`.

Estado real: el proyecto ya tiene un logger funcional (`src/infraestructura/logger.py`, rotación 10MB/5 backups, formato `asctime - name - levelname - message`) — es más simple que lo que pide esta US (sin JSON estructurado, sin `correlation_id`, sin redacción de secretos, sin Seq). Es un buen punto de partida, no un reemplazo completo de la US-107. Ver [docs/info_modulo/01-logger.md](#).

Épica H1-E3: Entorno de Calidad y Pipeline CI/CD

US-108 — Entorno de Desarrollo y Pipeline CI/CD

- **Esfuerzo:** S (1-2 días) · **Prioridad:** Alta · **Dependencias:** —
- **Objetivo Funcional:** garantizar que todo nuevo commit sea verificado automáticamente con linting, tests y cobertura, haciendo del pipeline la única fuente de verdad sobre el estado de calidad del proyecto.
- **Archivos a crear/existentes:**

<code>.github/workflows/linter.yml</code>	✓ existe (ruff, reglas E + I)
<code>.github/workflows/test.yml</code>	✓ existe (pytest + cobertura, matriz Linux/Windows)
<code>pytest.ini</code>	✓ existe (pythonpath=src, testpaths=test)
<code>.pre-commit-config.yaml</code>	✓ existe (check-yaml, end-of-file-fixer, trailing-whitespace, black)
<code>pyproject.toml</code>	✓ existe (config de ruff – falta version/entrypoint del paquete)
<code>Makefile</code>	✓ existe (instalar_dependencias_w/l, run_test, run_linter_ruf, corregir_linter, pre_commit)
<code>docs/catalogo-criticidad.md</code>	✗ no existe – ver sección 14
<code>docs/guias/01-flujo-de-trabajo-git.md</code>	✗ no existe

- **Criterios de Aceptación:**

- **AC1.** `make test/run_test` ejecuta la suite completa y **debería** fallar si la cobertura es $< 80\%$ en módulos no críticos — hoy no lo hace (ver sección 20).
- **AC2.** `make lint/run_linter_ruf` falla ante cualquier infracción de estilo.
- **AC3.** El pipeline CI falla el PR ante test fallido, cobertura insuficiente o error de linting.
- **AC4.** `pre-commit install` configura hooks locales en un único comando.
- **AC5.** `docs/catalogo-criticidad.md` inicializado con al menos los módulos de autenticación y persistencia.
- **Testing Mínimo:** manual — push a rama feature dispara el workflow; error de lint intencional hace fallar CI; cobertura por debajo del umbral hace fallar CI con mensaje explicativo.

Estado real y propuestas de ampliación: ver `docs/ideas-aprendizaje.md` sección 7 ("Completar el pipeline de CI") para 8 mejoras concretas ya identificadas (cobertura que bloquee de verdad, `mypy`, `ruff format --check`, `pip-audit`, `gitleaks`, Dependabot, build de Docker en CI). No repetido acá para no duplicar contenido.

10. Hito 2 — Motor de Ingesta y Análisis (v0.2)

Objetivo del hito: automatizar la carga desde Ges Deportivo, producir métricas avanzadas confiables, exponer estadísticas por CLI y asegurar la evolución controlada del esquema de base de datos.

Épicas: 3 · **Historias:** 5 · **Esfuerzo total estimado:** ~30 días·persona

Decisión previa requerida: ADR-002 debe estar aprobado (framework UI: Flet vs. Compose Multiplatform) — *nota: el LaTeX dice que ADR-002 bloquea Hito 2, pero la tabla de ADRs (sección 8) lo lista bloqueando Hito 4; se transcribe la referencia tal cual aparece en cada fuente como otro punto a resolver por el equipo, ver sección 20.*

Épica H2-E1: Integración "Ges Deportivo"

US-201 — Parseo de Planillas Excel con Pandas

- **Esfuerzo:** L (6-10 días) · **Prioridad:** Urgente (bloqueante) · **Dependencias:** US-105, ADR-003
- **Objetivo Funcional:** convertir planillas Excel externas de Ges Deportivo en datos persistentes sin transcripción manual, incluyendo modo *preview* sin efecto secundario y reporte de calidad de la importación.
- **Narrativa:** Como analista, quiero procesar los archivos de Ges Deportivo para eliminar el error humano en la transcripción y agilizar el análisis.
- **Capa de Dominio: Excepciones:** `InvalidExcelFormatError`.
- **Capa de Aplicación:**
 - **Caso de uso:** `ImportarExcelUseCase`.
 - **DTOs:** `IngestRowDTO`, `IngestResultDTO`, `ResultadoImportacionDTO`, `IngestReportDTO` (contiene `creados`, `actualizados`, `rechazados` con causa, y totales).
- **Capa de Infraestructura:**
 - **Servicios:** `GesDeportivoExcelParser` (`src/infraestructura/ingest/excel_parser.py`) — lee con `pd.read_excel()`, mapea columnas al formato interno, convierte tipos, detecta filas malformadas; `IngestService` (`src/infraestructura/ingest/ingest_service.py`) — lógica de merge y validación cruzada.
 - **CLI:** `stats import excel --file <ruta>`.
- **Base de Datos:** afecta `club`, `jugador`, `competencia`, `partido`, `jugadorPartido`.

- **Criterios de Aceptación:**

- **AC1 — Mapeo y Validación de Formato:** el parser lanza `InvalidExcelFormatError` si faltan columnas requeridas, antes de procesar.
- **AC2 — Lógica de Merge:** si el jugador no existe (por DNI), se crea automáticamente; si ya existe, se vincula.
- **AC3 — Verificación de Consistencia:** la suma de puntos individuales debe coincidir con el resultado final del partido cargado. **Este AC no es implementable tal cual está redactado con el schema actual:** no existe ningún campo que guarde el "resultado final" oficial del partido para comparar contra la suma de `jugadorPartido.puntos` — hoy solo se puede sumar el boxscore contra sí mismo, no contra un total independiente informado por la planilla de Ges Deportivo. Se necesita primero agregar a `Partido/partido` un campo de resultado final (ej. `puntosLocalFinal/puntosVisitanteFinal` — propuesta ya volcada en el DER de `docs/diagramas/diagramas.md` y en la sección 20) antes de poder escribir este AC como un test verificable.
- **AC4 — Transaccionalidad:** el proceso es atómico por partido; si falla una estadística, no se guarda el partido ni sus jugadores asociados.
- **AC5 — Vista Previa:** método `preview()` que valida el archivo sin persistir cambios.
- **AC6 — Logs:** reporte con registros procesados, jugadores creados/vinculados y errores.

- **Reglas de Negocio:**

- Fila sin DNI en el Excel → se rechaza (log y continuar).
- Club o competencia inexistentes → se crean automáticamente con los nombres provistos.
- El formato de fechas se valida según el estándar del proyecto.

- **Testing Mínimo:**

- *Unitarias:* mapeo de columnas, tipado correcto, detección de errores de formato; estrategias de merge y rechazo de filas inválidas.
- *Integración:* importación de fixture a DB `:memory:`, verificando persistencia y rollback ante error; modo `preview` no produce cambios.

Archivos a crear:

```
src/infraestructura/ingest/
├── excel_parser.py
└── ingest_service.py

src/aplicacion/
├── use_cases/importar_excel.py
└── dtos/ingest_dto.py

test/
├── test_excel_import.py (integración)
└── fixtures/ges_deportivo_sample.xlsx (con casos borde: fila sin DNI, puntos inconsistentes, jugador ya existente)
```

Épica H2-E2: Motor Estadístico Avanzado

US-202 — Cálculo de Métricas Avanzadas

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Alta · **Dependencias:** US-201
- **Objetivo Funcional:** implementar el motor lógico de analítica deportiva para transformar datos crudos del boxscore en indicadores avanzados de rendimiento (eficiencia, porcentajes ajustados, ritmos de juego), mediante funciones puras determinísticas.
- **Narrativa:** Como DT, quiero ver métricas como eFG%, EFF, PPP y PER para evaluar el impacto real de mis jugadores y comparar el rendimiento de mi equipo contra el rival en cada partido.
- **Capa de Dominio: Excepciones:** `CalculationError` (datos inconsistentes, ej. lanzamientos negativos).
- **Capa de Aplicación:**
 - **Casos de uso:** `CalcularEstadisticasAvanzadasUseCase` (aplica fórmulas sobre DataFrames), `GenerarTablaComparativaUseCase` (agrupa por club, "Equipo vs Rival").
 - **DTOs:** `MetricasAvanzadasDTO`, `ComparativaEquipoDTO`, `MetricasDTO`.
- **Capa de Infraestructura:** `src/infraestructura/analytics/formulas.py` — funciones puras que reciben un `DataFrame` y retornan serie/escalar; **sin acceso a DB, sin efectos secundarios.**
- **Fórmulas mínimas requeridas:**
 - **eFG% (Effective Field Goal Percentage):** $(T2C + 1.5 \times T3C) / (T2L + T3L)$.
 - **EFF (Efficiency Index):** $PTS + REB + AST + REC + TAP_R - (T2L - T2C) - (T3L - T3C) - (T1L - T1C) - PERD$.
 - **PPP (Puntos por Posesión):** `Puntos / Posesiones`.
 - **Posesiones (estimación FIBA):** $(T2L + T3L) + 0.44 \times T1L + PERD - REB_OF$.
 - **% de Rebotes:** proporción de rebotes totales capturados sobre el total disponible.
- **Criterios de Aceptación:**
 - **AC1.** Disponibilidad de eFG%, EFF, PPP, PER (simplificado) y % de Rebotes.
 - **AC2.** Reporte comparativo "Equipo vs Rival" para un `idPartido` dado.
 - **AC3.** Los cálculos aceptan parámetros de filtro por Temporada e `idCompetencia` directamente en los DataFrames.
 - **AC4.** División por cero → 0.0; exclusión de `NaN` en resultados finales.
 - **AC5.** Cada fórmula documentada con su fuente/referencia técnica.
 - **AC6.** `formulas.py` no contiene ningún acceso a base de datos ni I/O externo.
- **Testing Mínimo:**
 - *Unitarias* (`test_formulas.py`): cobertura del **100%** de las funciones matemáticas, con valores calculados a mano y casos límite (ceros).
 - *Integración:* `GenerarTablaComparativaUseCase` con datos de dos equipos en un mismo partido (semilla), verificando que los totales coinciden con el resultado final.

Archivos a crear:

```
src/infraestructura/analytics/formulas.py

src/aplicacion/
├── use_cases/calcular_estadisticas_avanzadas.py
├── use_cases/generar_tabla_comparativa.py
└── dtos/metricas_dto.py

test/test_formulas.py
```


US-203 — Integración de Motor Estadístico (Pandas Engine)

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Alta · **Dependencias:** US-202
- **Objetivo Funcional:** conectar las Vistas SQL con DataFrames de Pandas para calcular métricas avanzadas automáticamente a partir de los datos cargados, sin acoplar fórmulas y persistencia.
- **Capa de Dominio: Interfaz:** `AnalyticsService`
(`src/dominio/repositorios/analytics_service.py`) — declara `get_boxscore_partido(partido_id)`, `get_totales_temporada(club_id, temporada)`.
- **Capa de Aplicación:**
 - **Caso de uso:** `CalcularEstadisticasPartidoUseCase`.
 - **DTOs:** `MetricasPartidoDTO`, `MetricasJugadorDTO`.
- **Capa de Infraestructura:** `PandasAnalyticsService`
(`src/infraestructura/analytics/pandas_analytics_service.py`) — lee desde las vistas SQL vía `pandas.read_sql()`, aplica las fórmulas de `formulas.py`, retorna DataFrames con columnas estandarizadas.
- **Vistas SQL requeridas:** `v_boxscore_completo`, `v_jugador_totales_temporada`.
- **Criterios de Aceptación:**
 - **AC1 — `formulas.py` puro:** ninguna función accede a la DB; todas aceptan `pd.DataFrame` y retornan resultados. Cobertura 100%.
 - **AC2 — División por cero protegida:** retorna 0.0 (no NaN/inf) en casos límite.
 - **AC3 — Uso de Vistas SQL:** `AnalyticsService` lee exclusivamente de las vistas, nombres de columnas consistentes.
 - **AC4 — Integridad de Datos:** porcentajes expresados como float entre 0-100 o como ratio según corresponda.
- **Testing Mínimo:**
 - *Unitarias:* fórmulas con DataFrames en memoria.
 - *Integración:* vistas reales en DB `:memory:` con datos semilla, comparadas con valores esperados.
 - *Regresión:* test que falla si se renombra una columna consumida.

US-205 — Consulta Estadística por CLI

- **Esfuerzo:** S (1-2 días) · **Prioridad:** Media · **Dependencias:** US-203, US-106
- **Objetivo Funcional:** exponer las métricas del motor analítico directamente desde la CLI, permitiendo al DT consultar líderes, estadísticas de un partido y comparativas de equipo sin necesidad de la interfaz gráfica.
- **Comandos CLI:**
 - `stats show partido <id>` — boxscore + métricas avanzadas del partido.
 - `stats leaders <temporada> [--top N]` — top-N por PTS, REB, AST, EFF.
 - `stats compare <partido_id>` — comparativa equipo vs. rival.
- **Criterios de Aceptación:**
 - **AC1.** Cada comando produce salida tabular legible con alineación numérica correcta.
 - **AC2.** Filtros `--temporada` y `--competencia` funcionan en `stats leaders`.
 - **AC3.** Partido inexistente → mensaje descriptivo, no traceback.
 - **AC4.** Todos los valores consistentes con los calculados por `formulas.py`.
- **Testing Mínimo:** integración — cada subcomando con datos semilla verificando columnas y valores; partido/temporada inexistente → mensaje de error controlado.

Archivos a crear (US-203 + US-205):

```
src/dominio/repositorios/analytics_service.py
src/infraestructura/analytics/pandas_analytics_service.py
src/aplicacion/use_cases/calcular_estadisticas_partido.py

src/infraestructura/ui/cli/commands/stats_commands.py
src/infraestructura/ui/cli/formatters/stats_formatter.py

test/
├─ test_analytics_service.py (integración)
├─ test_stats_commands.py (integración)
```

US-204 — Versionado de Esquema y Migraciones

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Alta · **Dependencias:** US-101, ADR-004
- **Objetivo Funcional:** asegurar la evolución controlada del esquema de base de datos entre versiones sin pérdida de datos, con un runner que aplique migraciones en orden y registre cada resultado.
- **Archivos a crear:**

```
src/infraestructura/persistencia/sql/migrations/
├─ 001_init.sql          ← schema completo de v0.1; desde acá el schema no se
toca directo
├─ 002_*.sql             ← migraciones incrementales, NNN_descripcion_corta.sql,
idempotentes

src/infraestructura/persistencia/migration_runner.py

test/test_migrations.py (upgrade multi-versión y rollback)
docs/guias/03-como-agregar-una-migracion.md
```

- **Criterios de Aceptación:**
 - **AC1.** Tabla `schema_version` creada y mantenida automáticamente (número de versión, nombre, timestamp).
 - **AC2.** El runner ejecuta solo migraciones pendientes (idempotente en las ya aplicadas).
 - **AC3.** Soporte de rollback controlado para la última migración aplicada (script `down` opcional).
 - **AC4.** Si el schema es incompatible con la versión del código al arrancar, la ejecución se bloquea con mensaje claro.
 - **AC5.** Scripts versionados con el mismo estándar de nomenclatura, registrados en el changelog técnico.
- **Testing Mínimo:** upgrade multi-versión sobre base con datos reales (cardinalidad conservada); rollback de la última migración (integridad post-rollback); dataset de versión anterior conserva todas las relaciones tras migrar.

11. Hito 3 — Visualización Pro y Reporting (v0.3)

Objetivo del hito: convertir las estadísticas en tableros, gráficos e informes consumibles por el DT para tomar decisiones tácticas antes, durante y después del partido.

Épicas: 3 · **Historias:** 3 · **Esfuerzo total estimado:** ~22 días·persona

Decisiones previas requeridas: ADR-005 (librería PDF), ADR-007 (motor de visualización).

Hallazgo de transcripción: el `.md` del submódulo solo tenía **US-301 y US-302** para este hito — **le faltaba por completo la Épica H3-E3 (Scouting) con US-303**, que sí está en el LaTeX. Se completa acá con la fuente que sí la tiene. Ver sección 20.

Épica H3-E1: Dashboards e Informes (CLI & Engine)

US-301 — Dashboards e Informes Interactivos

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Alta · **Dependencias:** US-203
- **Objetivo Funcional:** proveer una visualización avanzada de datos en la terminal y preparar el motor de generación de gráficos para la futura GUI.
- **Narrativa:** Como DT, quiero ver tablas de líderes y gráficos de tendencia en mi terminal para analizar el rendimiento del equipo sin salir de la CLI.
- **Capa de Aplicación:**
 - **Casos de uso:** `ObtenerLideresTemporadaUseCase`, `GenerarGraficoRendimientoUseCase`.
 - **DTOs:** `LiderDTO`, `GraficoDTO`.
- **Capa de Infraestructura:**
 - **Reportería CLI:** `TablaLideresReporter` (usa `rich`), `GraficoTendenciaReporter` (usa `textual` o `rich.panel`).
 - **Motor de Gráficos:** `ChartGenerator` (`src/infraestructura/analytics/chart_generator.py`) — genera figuras a partir de `DataFrames`, según la librería que defina ADR-007.
- **Vistas SQL requeridas:** `v_jugador_totales_temporada`, `v_partidos_resumen`.
- **Criterios de Aceptación:**
 - **AC1 — Dashboards en Consola:** uso de `rich` para mostrar top-5 de líderes por rubro (puntos, rebotes, EFF).
 - **AC2 — Generación de Figuras:** `ChartGenerator` produce gráficos (PNG o interactivos) en **menos de 2 segundos** para un dataset de referencia (3 temporadas, 20 equipos, 1200 filas de `boxscore`).
 - **AC3 — Interactividad:** filtro por temporada (`--season 2025`) en los comandos de reportes.
 - **AC4.** Valores graficados coinciden con los calculados por las vistas SQL.
- **Implementación requerida:** `stats leaders --season 2025` invoca al caso de uso correspondiente y muestra resultados formateados.
- **Testing Mínimo:**
 - *Unitario:* ordenamiento de líderes y desempate por criterio secundario; transformación de `DataFrame` a serie temporal para gráficos.
 - *Integración:* reporter CLI + chart generator con datos semilla, verificando salida completa.

Épica H3-E2: Reportería y Exportación

US-302 — Exportación Profesional a PDF

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Alta · **Dependencias:** US-301, ADR-005
- **Objetivo Funcional:** emitir un reporte formal de partido o temporada en formato PDF para análisis y difusión interna del cuerpo técnico.
- **Narrativa:** Como DT, quiero exportar el boxscore a PDF para compartirlo con mi cuerpo técnico o imprimirlo.
- **Capa de Aplicación:**
 - **Caso de uso:** `ExportarReporteUseCase`.
 - **DTOs:** `ExportarReporteDTO`, `ReporteResponseDTO`.
- **Capa de Infraestructura:** `PDFReportGenerator` (`src/infraestructura/reports/pdf_generator.py`), librería según ADR-005.
- **Criterios de Aceptación:**
 - **AC1 — Formato Profesional:** encabezado con nombres de clubes, fecha, metadata; tablas de boxscore y métricas avanzadas con alineación numérica y cabeceras claras.
 - **AC2 — Convención de Nombres:** `boxscore_YYYY-MM-DD_Local_vs_Visitante.pdf`.
 - **AC3 — Robustez:** creación automática del directorio `exports/` si no existe; errores de I/O (ruta sin permisos, disco lleno) producen mensaje de recuperación, no crash.
- **Testing Mínimo:** generación exitosa de un PDF completo desde un `idPartido` con datos semilla (válido, no corrupto, abre con lector estándar); nombre de archivo y ruta esperados; ruta sin permisos → excepción controlada.

Archivos a crear (US-301 + US-302):

```
src/aplicacion/use_cases/
├── obtener_lideres_temporada.py
├── generar_grafico_rendimiento.py
└── exportar_reporte.py

src/aplicacion/dtos/reporte_dto.py

src/infraestructura/analytics/chart_generator.py
src/infraestructura/reports/
├── cli_leaderboard_reporter.py
└── pdf_generator.py

test/
├── test_leaderboard.py
├── test_chart_generator.py (integración)
└── test_pdf_export.py (integración)
```

Épica H3-E3: Inteligencia de Scouting

US-303 — Scouting de Rival Pre-Partido

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Media · **Dependencias:** US-203
- **Objetivo Funcional:** entregar al DT un reporte táctico detallado del rival antes del encuentro, basado en el historial de partidos de la temporada.
- **Capa de Infraestructura:** `src/infraestructura/persistencia/sql/views_scouting.sql` — vistas de análisis histórico por rival.

- **Capa de Aplicación:**
 - **Caso de uso:** `GenerarScoutingRivalUseCase`.
 - **DTOs:** `ScoutingDTO`.
- **Criterios de Aceptación:**
 - **AC1.** Ventana de análisis de últimos N partidos configurable por parámetro.
 - **AC2.** Detección de patrones: rachas (victorias/derrotas consecutivas), tendencia de tiro, pérdidas recurrentes.
 - **AC3.** Filtros por competencia y categoría producen resultados consistentes con las vistas SQL.
 - **AC4.** Reporte exportable (texto/PDF) con conclusiones destacadas y datos fuente citados.
- **Testing Mínimo:**
 - *Unitario:* agregaciones históricas y funciones de detección de rachas.
 - *Integración:* vistas de scouting con dataset histórico de ejemplo (mínimo 10 partidos del rival); consistencia de filtros por competencia y ventana N .

Archivos a crear:

```
src/infraestructura/persistencia/sql/views_scouting.sql
src/aplicacion/use_cases/generar_scouting_rival.py
src/aplicacion/dtos/scouting_dto.py
test/test_scouting_rival.py (integración)
```

12. Hito 4 — Interfaz Multiplataforma y Entrega (v1.0)

Objetivo del hito: migrar a experiencia visual completa con Flet, cerrar el hardening de seguridad, garantizar continuidad operativa mediante backup/restore y distribuir la aplicación como binario autónomo para las plataformas objetivo.

Épicas: 4 · **Historias:** 4 · **Esfuerzo total estimado:** ~35 días·persona

Decisión previa requerida: ADR-002 (Flet confirmado como framework de UI).

Hallazgo de transcripción: el `.md` del submódulo solo tenía **US-401** para este hito — **le faltaban por completo las épicas H4-E2 (Resiliencia/Backup, US-402), H4-E3 (Seguridad, US-403) y H4-E4 (Empaquetado, US-404)**, que sí están en el LaTeX. Se completa con esa fuente. Ver sección 20.

Épica H4-E1: Interfaz de Usuario Adaptable (GUI)

US-401 — Implementación de Interfaz Flet (Desktop/Mobile)

- **Esfuerzo:** L (6-10 días) · **Prioridad:** Alta · **Dependencias:** US-301, US-302, ADR-002
- **Objetivo Funcional:** ofrecer una interfaz visual completa reutilizando los casos de uso consolidados en hitos previos, operativa en desktop y mobile sin modificar la lógica de negocio.
- **Narrativa:** Como DT, quiero una experiencia fluida y visual que no requiera comandos para gestionar mis estadísticas desde mi PC o celular.
- **Capa de Infraestructura (UI):**
 - **Tecnología:** Flet (Python-based).
 - **Componentes:** `ChartComponent` (wrapper de los gráficos de US-301), `StatsTableComponent`.

- **Navegación/Pantallas:** `DashboardScreen`, `GameEntryScreen`, `PlayerProfileScreen`, `ImportScreen`.
- **Criterios de Aceptación:**
 - **AC1 — Performance:** tiempo de arranque < **3 segundos** (desktop, 4GB RAM, SSD, Python 3.11+).
 - **AC2 — UX:** soporte de Modo Oscuro/Claro basado en preferencias del sistema.
 - **AC3 — Responsividad:** adaptación automática a resoluciones de PC (1080p) y Mobile (720p).
 - **AC4 — Gestión de Datos:** botón de "Sincronización/Backup" para exportar el `.sqlite` manualmente.
 - **AC5.** Formularios con validación visual (mensajes de error en campo, sin modal genérico).
 - **AC6.** La UI no contiene lógica de negocio; invoca casos de uso vía inyección de dependencias.
- **Testing Mínimo:**
 - *Unitario:* validadores de formularios, reglas de navegación de estado de pantalla.
 - *Manual guiado:* flujo Dashboard → GameEntry → PlayerProfile en desktop y mobile; importación de Excel desde la pantalla de importación.

Archivos a crear:

```
src/infraestructura/ui/flet/  
├─ app.py  
├─ screens/  
│   ├── dashboard_screen.py  
│   ├── game_entry_screen.py  
│   ├── player_profile_screen.py  
│   └─ import_screen.py  
└─ components/  
    ├── chart_component.py  
    └─ stats_table_component.py  
  
test/test_flet_validators.py
```

Épica H4-E2: Resiliencia y Backup

US-402 — Backup y Restauración de Base de Datos

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Alta · **Dependencias:** US-204, ADR-008
- **Objetivo Funcional:** proteger la continuidad operativa mediante un mecanismo de exportación e importación de la base de datos completa, con verificación automática de integridad post-restauración.
- **Criterios de Aceptación:**
 - **AC1.** Exportación completa incluye metadata: versión del schema, fecha/hora, hash de integridad.
 - **AC2.** Restauración funciona tanto sobre instancia vacía como sobre instancia con datos existentes.
 - **AC3.** Verificación automática post-restore: conteo de filas en tablas críticas y checksum lógico de datos clave.
 - **AC4.** Checklist de estabilización ejecutada y documentada (rendimiento, errores críticos, regresión).

- **Testing Mínimo:** ciclo completo exportar → restaurar → verificar igualdad de datos clave; restauración desde backup de versión anterior con migraciones aplicadas automáticamente; regresión de casos críticos tras restore (auth, carga partido, exportación PDF).

Archivos a crear:

```
src/aplicacion/use_cases/  
├─ exportar_backup.py  
└─ restaurar_backup.py  
  
src/infraestructura/persistencia/backup_service.py  
test/test_backup_restore.py (integración)
```

Épica H4-E3: Seguridad de Datos Local

US-403 — Hardening de Seguridad Local

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Alta · **Dependencias:** US-104, ADR-006
- **Objetivo Funcional:** endurecer credenciales, política de sesión y almacenamiento local para que la aplicación cumpla con un nivel de seguridad adecuado al contexto de datos deportivos personales.
- **Criterios de Aceptación:**
 - **AC1.** Contraseñas mínimo **12 caracteres**, con validación de complejidad (mayúscula, número, símbolo).
 - **AC2.** Lista local de contraseñas comprometidas bloquea efectivamente su uso en registro y cambio.
 - **AC3.** Procedimiento de actualización mensual documentado en [docs/security/](#), con pasos reproducibles.
 - **AC4.** CI valida versión vigente y checksum de la lista de comprometidas en cada PR.
 - **AC5.** Hashing robusto (bcrypt o Argon2) con salt único por usuario; migración de hashes legados.
 - **AC6.** Cifrado de base de datos local configurable (SQLCipher) sin romper la API del [DatabaseManager](#).
 - **AC7.** Política de sesión: expiración configurable, revocación inmediata en logout, limpieza idempotente.
 - **AC8.** Eventos de seguridad (fallos de login, cambios de contraseña) trazados en log sin exponer datos sensibles.
- **Testing Mínimo:** política de contraseñas (longitud, complejidad, lista comprometidas); ciclo hash → verificación, migración de hash legado; expiración y revocación de sesión; acceso a DB con y sin cifrado según configuración.

Archivos a crear:

```
src/infraestructura/security/  
├─ credential_policy.py  
├─ compromised_password_store.py  
└─ db_encryption_adapter.py (adaptador SQLCipher, opcional)
```

```
docs/security/password_compromised_update_procedure.md
test/test_credential_policy.py
```

Épica H4-E4: Empaquetado y Distribución

US-404 — Empaquetado y Distribución Multiplataforma

- **Esfuerzo:** M (3-5 días) · **Prioridad:** Alta · **Dependencias:** US-401, US-108
- **Objetivo Funcional:** generar binarios autónomos distribuibles para cada plataforma objetivo (Windows, Linux, macOS, Android) sin requerir instalación de Python ni dependencias externas por parte del usuario final.
- **Criterios de Aceptación:**
 - **AC1.** Binario desktop lanza la aplicación sin Python instalado en el sistema del usuario.
 - **AC2.** APK de Android instalable en dispositivos Android 9+ sin dependencias adicionales.
 - **AC3.** El pipeline de release genera artefactos para las 3 plataformas desktop en cada tag de versión.
 - **AC4.** Versión embebida en la aplicación (About screen / CLI `--version`) coincide con el tag de release.
 - **AC5.** `CHANGELOG.md` actualizado siguiendo formato *Keep a Changelog* en cada release.
- **Testing Mínimo:** instalar binario desktop en máquina limpia (sin Python) y ejecutar flujo completo; instalar APK en dispositivo/emulador Android y verificar arranque; el pipeline de release ejecuta tests completos antes de generar artefactos.

Archivos a crear:

```
build/
├─ build_desktop.py    ← script PyInstaller (Windows/Linux/macOS)
├─ build_android.py    ← flet build apk
└─ build_ios.py        ← flet build ipa (requiere macOS + Xcode)

.github/workflows/release.yml    ← se activa con tag v*.*.*
CHANGELOG.md                    ← formato Keep a Changelog
docs/install-guide.md           ← instrucciones de instalación por plataforma
```

13. Definición de "Hecho" (DoD)

Nota de transcripción importante: las dos fuentes tenían **tres versiones distintas** de la Definición de "Hecho" (una en el LaTeX, y **dos** dentro del mismo `.md` del submódulo — una etiquetada "v2" a mitad de documento y otra al final, más corta y parcialmente contradictoria entre sí). Se consolidan en una sola versión, la más completa, marcando esto como hallazgo en la sección 20.

Una Historia de Usuario se considera **Hecha** cuando cumple TODOS los siguientes puntos:

1. **Código integrado:** merge a la rama principal sin conflictos; commits con formato `tipo(alcance): descripción`.

2. **Tests pasan:** cobertura $\geq 80\%$ en lógica de negocio general; $\geq 95\%$ en componentes críticos (autenticación, persistencia transaccional, rollback, fórmulas estadísticas — ver Catálogo de Criticidad). Todos los tests corren sin errores en CI.
3. **Arquitectura respetada:** ninguna clase en `dominio/` importa de `infraestructura/` ni de librerías externas (`sqlite3`, `pandas`). Verificable con `grep` o una herramienta de análisis de imports.
4. **SQL verificado:** si la US crea o modifica una vista SQL, la vista está en `views.sql` y existe un test de integración que la consulta con datos de `seed.sql`.
5. **Transacciones:** si la US persiste múltiples tablas (ej. partido + estadísticas), existe un test que verifica que un fallo parcial hace rollback completo.
6. **ADR documentado:** si la US requirió una decisión arquitectónica, el ADR correspondiente está aprobado y en el repositorio.
7. **Sin imports cruzados:** verificado con una herramienta de linting de arquitectura.
8. **Docstring mínimo:** todos los métodos públicos tienen docstring de una línea; los métodos complejos documentan sus parámetros.
9. **Si incluye UI:** validada manualmente en al menos dos plataformas (desktop + mobile).
10. **Trazabilidad:** eventos de log relevantes implementados con `correlation_id` (ver US-107).
11. **Pipeline CI en verde:** lint + tests + cobertura.

El repositorio del agregado tiene su propia interfaz en el dominio, y la persistencia fue validada mediante una prueba de integración con SQLite son, además, dos condiciones que ambas fuentes repiten como base mínima innegociable — se mantienen implícitas en los puntos 3 y 4 de arriba.

14. Catálogo Técnico de Criticidad

Artefacto vivo del backlog de arquitectura donde se lista cada módulo con: nivel de criticidad, justificación, owner técnico y cobertura objetivo.

Ubicación: `docs/catalogo-criticidad.md` (✗ no existe todavía — se inicializa en US-108).

Estructura mínima de la tabla: `Módulo` | `Criticidad` | `Justificación` | `Owner` | `Cobertura objetivo` | `Evidencia`.

Criterios de clasificación (crítico si cumple al menos uno):

- Controla acceso, autenticación o autorización.
- Persiste datos en múltiples tablas con transacciones.
- Impacta la integridad histórica del dato deportivo.
- Implementa cálculos estadísticos oficiales usados en decisiones tácticas.

Gobernanza: owner primario Arquitecto Técnico; co-owner QA Lead. Revisión obligatoria en kickoff de hito y en cada planificación de sprint.

15. Proceso de Liberación de Versiones

Versionado semántico

`vMAJOR.MINOR.PATCH` (Semantic Versioning 2.0):

- **MAJOR** — cambios incompatibles con versiones anteriores (ej. rediseño del schema que requiere migración manual).
- **MINOR** — nuevas funcionalidades compatibles. **Cada hito completado incrementa el MINOR** (H1 → v0.1.0, H2 → v0.2.0, H4 → v1.0.0).
- **PATCH** — corrección de bugs sin nuevas funcionalidades (ej. v0.1.1).

Mientras **MAJOR** = 0, la API no se considera estable — cualquier hito puede cambiar el schema o los contratos internos. La versión **1.0.0** se alcanza al completar el Hito 4.

Estrategia de ramas

Rama	Propósito
main	Solo commits de merge desde release/ . Cada commit tiene un tag de versión. Nunca se trabaja directo acá.
develop	Rama de integración; las feature/ se mergean acá.
feature/nombre	Una rama por US o tarea, desde develop , mergeada por PR. Ej. feature/us-101-sqlite-schema .
release/vX.Y.Z	Se crea desde develop cuando el hito está feature-complete. Solo acepta fixes y bump de versión.
hotfix/descripcion	Para bugs críticos en producción; desde main , se mergea a main y a develop .

Proceso de release paso a paso

1. **Preparar la rama:** `git checkout -b release/v0.1.0 develop`; actualizar **version** en `pyproject.toml`; mover `[Unreleased]` a `[0.1.0] - YYYY-MM-DD` en `CHANGELOG.md`; correr `make test` (debe pasar); solo corregir bugs críticos, sin features nuevas.
2. **Mergear a main y taggear:** `merge --no-ff`; el tag describe el contenido del hito, no solo el número; `git push origin main --tags`.
3. **Mergear de vuelta a develop:** para que los fixes del release lleguen a desarrollo; borrar la rama de release.
4. **Crear el GitHub Release:** con `gh release create v0.1.0 --title "..."` `--notes-file CHANGELOG.md`; adjuntar binarios con `gh release upload`; o automatizado por `.github/workflows/release.yml` (US-404) al detectar el tag.
5. **Verificar el release:** confirmar tag/notas/artefactos en GitHub; instalar el binario en máquina limpia y correr el smoke-test de instalación.

Formato del CHANGELOG.md

Estándar **Keep a Changelog** (keepachangelog.com): sección `[Unreleased]` al tope para cambios en desarrollo; una sección `[vX.Y.Z] - YYYY-MM-DD` por release con subsecciones **Added**, **Changed**, **Fixed**, **Removed**, **Security**.

Buena práctica: actualizar `[Unreleased]` en cada PR mergeado, no solo al momento del release.

Hotfix — corregir un bug en producción

1. `git checkout -b hotfix/descripcion-breve main`.
2. Aplicar la corrección mínima; un test de regresión que falla sin el fix.
3. Actualizar `PATCH` en `pyproject.toml` (ej. 0.1.0 → 0.1.1) y `CHANGELOG.md`.
4. Merge `--no-ff` a `main`; tag `v0.1.1`.
5. Mergear también a `develop` para no perder el fix.
6. Crear GitHub Release con la nota del bug corregido.

16. Roadmap Futuro (Hitos 5–9, visión de producto)

Los hitos 5–9 son **visión de producto, no compromisos de entrega**. Se refinan y priorizan según feedback real de usuarios al cierre de v1.0. Se transcriben completos porque forman parte del PRD original, aunque no son parte del plan de trabajo actual.

- **Hito 5 · Ecosistema Conectado y Colaborativo (v1.5):** romper el aislamiento local. Sincronización cloud híbrida (FastAPI + PostgreSQL, manteniendo la app 100% funcional offline); exportación de "Match Cards" para redes sociales; base de datos compartida de scouting entre DTs (con consentimiento explícito).
- **Hito 6 · Inteligencia Táctica con IA (v2.0):** pasar de análisis descriptivo a prescriptivo. Motor de recomendación de quinteto (scikit-learn/XGBoost); detección automática de patrones (rachas de tiro, degradación defensiva por fatiga, pérdidas en momentos de presión — z-score, ARIMA); explicabilidad con SHAP values; integración de video scouting; marco de experimentación A/B de estrategias.
- **Hito 7 · Live Stats y Cancha Digital (v3.0):** captura de datos en tiempo real. Módulo de captura en vivo en tablet (<200ms por acción); dashboard en vivo para el banco (PPP por posesión, eFG% acumulado, +/- por jugador); mapa de tiro interactivo por zonas; control de carga/fatiga con alertas.
- **Hito 8 · Plataforma Federativa y Competición Multi-Liga (v4.0):** escalar de club a federación. Integración con APIs federativas (CABB, FIBA); vista multi-club para oficiales de liga; reportería regulatoria automatizada; benchmarking anónimo inter-club; gestión de árbitros y planilleros.
- **Hito 9 · Multi-Deporte y Ecosistema de Formación (v5.0):** extender más allá del básquet. Motor estadístico multi-deporte configurable (voleibol, fútbol sala, handball); seguimiento de desarrollo a largo plazo entre categorías; modelado de riesgo de lesión por carga acumulada; módulo de formación para entrenadores; portal de seguimiento para familias (solo lectura, opt-in).

17. Estructura de Repositorios

Cada módulo de datos se divide siguiendo el S.R.P. (Single Responsibility Principle):

Entidad / Agregado	Repositorio	Vistas Relacionadas	Estado real
Identidad	<code>SqliteUsuarioRepositorio</code>	N/A	✅ funcional y testado
Clubes	<code>SqliteClubRepositorio</code>	<code>v_listas_detalle</code>	✅ funcional, sin tests dedicados
Jugadores	<code>SqliteJugadorRepositorio</code>	<code>v_jugador_totales_temporada</code>	⚠️ bug en <code>link_to_club</code>

Entidad / Agregado	Repositorio	Vistas Relacionadas	Estado real
			(ver sección 20)
Competencia	SqliteCompetenciaRepositorio	v_listas_detalle	⚠ implementado, con 3 bugs y 2 métodos sin implementar (ver sección 20)
Partidos/Stats	SqliteJuegoRepositorio	v_partidos_resumen, v_boxscore_completo	✗ no funcional (ver sección 20)

Corrección respecto a ambas fuentes originales: tanto el LaTeX como el .md del submódulo listan esta tabla con **solo 4 filas**, omitiendo el repositorio de Competencia — pese a que el propio texto de la US-102 en ambas fuentes lo describe en detalle. Es la inconsistencia original que motivó esta revisión completa del documento.

18. ADRs Pendientes — tabla de bloqueo por hito

(Duplicado intencional de la sección 8, en el formato de tabla de bloqueo que traían ambas fuentes, por si se prefiere consultar esta vista en vez de la de "Registro de Decisiones" 😊)

ADR	Título	Bloquea	Decisión recomendada
ADR-001	Arquitectura Local-First	Hito 1	✅ SQLite + offline-first (ya definido)
ADR-002	Framework UI	Hito 2 (LaTeX) / Hito 4 (tabla ADR)	Evaluar Flet vs. Compose Multiplatform — resolver la discrepancia de a qué hito bloquea, ver sección 20
ADR-003	Protocolo de Ingesta Excel	US-201/US-202	Pandas + investigación previa de columnas reales de Ges Deportivo
ADR-004	Versionado de DB	Hito 2	Migraciones manuales (<code>schema_version</code>) vs. <code>alembic</code>
ADR-005	Reportes PDF	US-302	<code>reportlab</code> (sin dependencias) o <code>weasyprint</code> (HTML→PDF)
ADR-006	Seguridad y Cifrado	US-106/US-403	v0.1: SHA-256 con salt fijo. v1.0: migrar a <code>bcrypt</code> con salt dinámico
ADR-007	Motor de Visualización	US-301	<code>matplotlib</code> (offline) o <code>plotly</code> (interactivo)

ADR	Título	Bloquea	Decisión recomendada
ADR-008	Estrategia de Backup	Hito 3 (<i>LaTeX</i>) / Hito 4 (<i>tabla ADR</i>)	Export manual de <code>.db</code> + script de restauración — misma discrepancia de hito, ver sección 20
ADR-009	Pipeline CI/CD	US-108	GitHub Actions para lint, tests y cobertura automáticos

19. Convenciones rápidas

- **Commits:** `tipo: descripción corta` (`FEAT`, `FIX`, `DOCS`, `STYLE`, `REFACTOR`, `PERF`, `TEST`) — ver `docs/info_modulo/02-reglas.md`. (El *LaTeX* propone el formato `tipo(alcance): descripción con alcance entre paréntesis` — ambos formatos conviven en las fuentes, el equipo debería fijar uno solo.)
- **Interfaces de repositorio:** el proyecto usa `ABC/@abstractmethod` en la práctica, **no** `typing.Protocol` como sugiere `docs/info_protocolos.md`. Ambas son válidas — vale un ADR corto para dejarlo asentado.
- **Logging:** `infraestructura/logger.py` ya implementado (rotación 10MB, 5 backups, nivel INFO+ a archivo) — ver `docs/info_modulo/01-logger.md`. Es la base sobre la que debería crecer la US-107 (JSON estructurado + `correlation_id`), no un reemplazo.
- **Análisis estático:** el proyecto usa `ruff` en la práctica (`pyproject.toml`, `.github/workflows/linter.yml`), no `flake8/pylint` como sugería el borrador original del Acuerdo de Ingeniería (sección 5) — ya reflejado arriba.

20. Estado real del código vs. plan (hallazgos)

Auditoría hecha releendo `src/` completo (dominio + infraestructura + tests), `schema.sql`, `views.sql`, `seed.sql`, `docs/diagramas/diagramas.md`, y corriendo la suite real (`pytest -q`) para esta revisión del 2026-08-09. Nada de esto se corrigió en código — es diagnóstico para que el equipo decida qué hacer.

Hallazgos de código (repositorios) — actualizado 2026-08-09

-  **Patrón de conexión — ya resuelto, corrección de un hallazgo anterior.** Una revisión previa de este documento decía que `SqliteJugadorRepositorio` y `SqliteJuegoRepositorio` seguían importando una clase `SqliteConexion` inexistente. **Ya no es así:** las 5 implementaciones (`usuario`, `club`, `jugador`, `competencia`, `juego`) reciben `sqlite3.Connection` **crudo** directamente en el constructor, de forma uniforme. Este punto queda cerrado.
-  **`SqliteCompetenciaRepositorio` ya no es un gap — existe, pero con bugs reales.** Cambia el diagnóstico de fondo: el archivo `src/infraestructura/repositorios/sqlite_competencia_repositorio.py` existe y tiene sus 12 métodos, pero:
 - `guardar_inscripcion`: el `INSERT` declara 3 columnas (`idClub`, `idCategoria`, `idCompetencia`) pero la query solo tiene **un ?** — `sqlite3.ProgrammingError` garantizado al ejecutarse.
 - `obtener_categorias`: `rows = cursor.fetchall` — falta el `()`; asigna el método en vez de invocarlo, y falla al intentar iterar sobre él.
 - `agregar_jugador_lista` y `obtener_jugadores_lista`: son stubs vacíos (`pass`) — no están implementados, pese a que la interfaz y la clase que los contiene ya existen. Sin esto, no se

puede poblar ni consultar quién está habilitado en una lista de buena fe a través del repositorio (hoy `seed.sql` lo hace con `INSERT` directo, saltando la capa de dominio).

- `obtener_lista_por_inscripcion`: tipada `-> list[ListaBuenaFe]` en la implementación, pero la interfaz de dominio la tipa `-> ListaBuenaFe` (singular) — y la relación real es 1:1 (`idInscripcion` `UNIQUE` en `listaBuenaFe`). La implementación contradice tanto su propia interfaz como la regla de negocio ya documentada en la sección 3.
- **✗ `sqlite_juego_repositorio.py` no es funcional (más allá del bug de tabla/columna ya conocido):**
 - `_row_to_entity` y `buscar_por_id` siguen usando `Juego/idJuego` (la tabla real es `partido`, la columna `idPartido`) — bug ya documentado en revisiones anteriores, sigue sin corregirse.
 - `guardar_partido` e inserta en la tabla inexistente `Juego`, y llama a `self.conexion.obtener_conexion()` — método que no existe sobre un `sqlite3.Connection` crudo `→ AttributeError` apenas se invoca.
 - `guardar_boxscore`: la lista de columnas del `INSERT` tiene 19 nombres (falta `idClub`) pero la tupla de valores tiene 20 `→ sqlite3.ProgrammingError` garantizado.
 - **Violación de Liskov — hallazgo nuevo:** la interfaz de dominio ya pide `guardar_partido(partido: Partido)` y `guardar_boxscore(boxscore: JugadorPartido)` (recibir la entidad completa), pero la implementación real sigue con parámetros sueltos (`fecha`, `estadio`, `idCompetencia`, ... y 19 parámetros respectivamente) — no cumple el contrato de la interfaz que dice implementar.
- **⚠ `SqliteJugadorRepositorio.link_to_club` — bug nuevo, no documentado antes:** usa `jc.id_jugador` y `jc.id_club`, pero la dataclass real `JugadorClub` tiene los campos `idJugador/idClub` (sin guión bajo) `→ AttributeError` garantizado al invocarlo. Además el método no retorna nada, pese a que la interfaz pide `-> JugadorClub`.
- **✅ `sqlite_usuario_repositorio.py` — sigue correcto:** `FROM usuario` e `idUsuario` correctos, mapeo `pw ↔ contrasenia` bien resuelto, y es el único repositorio con tests dedicados hoy.
- **⚠ Patrón sistémico — funciones que deberían devolver `list[X]` y devuelven `None`.** Es el mismo tipo de problema que se charló sobre pasar dataclasses en vez de parámetros sueltos, pero del lado de las lecturas: **todos** los métodos "listar"/"buscar_por_X" de las 5 implementaciones devuelven `None` cuando no hay resultados, en vez de `[]`, pese a que la interfaz de dominio los tipa `list[X]` sin `| None`. Rompe el contrato de tipos: cualquier código futuro que llame a estos métodos y haga `for x in resultado` va a explotar con `TypeError: 'NoneType' object is not iterable` si no se blindo contra `None` primero. Afecta: `ClubRepositorio.buscar_por_id_usuario`, `.buscar_por_nombre`; `JugadorRepositorio.buscar_por_club`; `JuegoRepositorio.buscar_por_club`; `CompetenciaRepositorio.obtener_todas_competencias`, `.obtener_categorias`, `.obtener_inscripciones_por_club`, `.obtener_lista_por_inscripcion`. Recomendación a documentar (no a aplicar): devolver `[]` en el `if not rows`, nunca `None`, para que el tipo de retorno sea siempre coherente con lo declarado en la interfaz.
- **⚠ Manejo de errores silencioso — patrón sistémico en todos los `guardar*`.** Las 5 implementaciones atrapan `sqlite3.Error` en sus métodos `guardar*` y devuelven `None` silenciosamente, sin loguear ni relanzar como excepción de dominio. Esto choca con las excepciones de negocio que el propio plan ya prevé (`DNIDuplicadoError`, `EmailYaRegistradoError`, etc. — sección "Reglas de Negocio Consolidadas", sección 6): hoy no hay forma de distinguir "guardado exitoso" de "falló por violar una regla real (ej. DNI duplicado)" de "falló por un bug" — las tres situaciones devuelven `None` igual. Cuando se construya la capa de aplicación (US-103 en adelante), este punto va a ser bloqueante para poder lanzar las excepciones de dominio que esas US ya dan por sentadas.

- **Typo cosmético que persiste:** la clase se llama `SquliteJugadorRepositorio` (falta una "l"). No rompe nada porque nada la importa por nombre todavía.
- **Divergencia menor de nombres de método:** el PRD (ambas fuentes) menciona `exists_by_email/UserRepository.exists_by_email` como parte del contrato de usuario; la interfaz real (`usuario_repositorio.py`) no lo tiene — solo `encontrar_por_mail`, `encontrar_por_id`, `guardar`. No es necesariamente un problema (se puede resolver llamando a `encontrar_por_mail` y chequeando `is not None`), pero vale la pena que el equipo decida si agregan el método explícito o lo dejan así.
- **Cobertura de tests real, verificada corriendo la suite hoy (`pytest -q`): 30 tests, 29 pasan, 1 falla** (`test_guardar_usuario_con_error`, porque las entidades siguen sin validar tipos en `__post_init__` — ya documentado más abajo en "Hallazgos de organización de código"). La suite se dividió desde la última revisión en 5 archivos por repositorio (`test_repositorios_{usuario,club,jugador,juego,competencia}.py`), pero **`test_repositorios_juego.py` y `test_repositorios_competencia.py` están completamente vacíos** — cero tests. Son justamente los dos repositorios con más bugs de los listados arriba; por eso el pipeline aparece en verde (salvo el único test que falla) pese a que ninguno de esos dos repositorios funcionaría hoy si se los usara.

Hallazgos del DER y diagramas (`docs/diagramas/diagramas.md`) — nuevo, 2026-08-09

Ya corregidos directamente en ese archivo (ver el DER y su nota de hallazgos ahí). Resumen:

- `categoria` tenía la PK mal nombrada (`idCompetencia` copiado por error en vez de `idCategoria`).
- La relación `inscripcion-listaBuenaFe` estaba dibujada 1:N (`||--o{`) cuando el schema real la fuerza 1:1 (`idInscripcion UNIQUE`).
- La relación `club-partido` estaba duplicada dos veces con la misma etiqueta, sin distinguir los roles local/visitante (dos FKs distintas: `idClubLocal`, `idClubVisitante`).
- La PK compuesta de `jugadorClub` en el diagrama le faltaba `fechaDesde` (el schema real es `PRIMARY KEY (idJugador, idClub, fechaDesde)`).
- Tipos de dato desalineados: `contraseña`→`contrasenia`, `dni` de `varchar`→`integer`, `partido.idCompetencia` de `varchar`→`integer`.
- El diagrama de clases tenía las firmas de los repositorios desactualizadas (parámetros sueltos en vez de las dataclasses que ya usa la interfaz real) — corregido para que coincida con `src/dominio/repositorios/*.py`.

Campo/tabla que falta para que el propio PRD sea implementable — nuevo, 2026-08-09

No existe ningún campo que guarde el resultado final oficial del partido

(ej. `puntosLocalFinal/puntosVisitanteFinal`). Importa porque la propia **US-201 AC3** pide verificar que "la suma de puntos individuales coincida con el resultado final del partido cargado", pero hoy no hay dónde guardar ese resultado final para comparar contra él — solo se puede sumar el boxscore contra sí mismo. Se dejó la propuesta volcada (marcada como propuesta, no implementada) en el DER de `docs/diagramas/diagramas.md` y anotada en US-105 y US-201 de este documento. Relacionado: tampoco existe todavía la tabla `schema_version` que pide US-204 (Hito 2, no arrancado — esperable, solo se deja anotado para cuando se aborde esa US).

Hallazgos de organización de código

- **Entidades agrupadas en un mismo archivo:** el PRD prevé un archivo por entidad (`categoria.py`, `inscripcion.py`, `lista_buena_fe.py`, etc. separados); el código real agrupa varias entidades relacionadas en un mismo archivo (ej. `competencia.py` contiene 5 dataclasses: `Competencia`, `Categoria`, `Inscripcion`, `ListaBuenaFe`, `JugadorListaBuenaFe`). Es razonable para el tamaño actual del proyecto, pero conviene un acuerdo explícito del equipo sobre si se mantiene así.
- **Capa de aplicación todavía no existe** (`src/aplicacion/`) — es esperable en este punto (Hito 1, US-103 en adelante no implementadas), no es un bug, solo un recordatorio de que `main.py` hoy no sigue el patrón de Composition Root descrito en la sección 4 porque todavía no hay casos de uso que orquestar.

Hallazgos de documentación (inconsistencias entre las fuentes del PRD)

- **El submódulo (`.md`) tenía Hito 3 y 4 incompletos.** Le faltaba la Épica H3-E3 (US-303, Scouting de Rival) completa, y las épicas H4-E2, H4-E3, H4-E4 (US-402 Backup, US-403 Seguridad, US-404 Empaquetado) completas. Solo estaban en el LaTeX. Ya se completó en este documento usando esa fuente.
- **Tres versiones distintas de la Definición de "Hecho" (DoD).** Una en el LaTeX (la más completa, con Catálogo de Criticidad integrado) y **dos** dentro del mismo archivo del submódulo (una "v2" a mitad de documento, otra más corta al final). Se consolidaron en la sección 13 de este documento, usando la más completa como base.
- **El `.md` del submódulo no tenía sección de Requisitos No Funcionales (NFR) en absoluto** — solo estaba en el LaTeX. Se agregó en la sección 7.
- **El `.md` del submódulo no tenía el Proceso de Liberación de Versiones** (versionado semántico, estrategia de ramas, pasos de release, formato de changelog, hotfix) — solo estaba en el LaTeX. Se agregó en la sección 15.
- **ADR-002 y ADR-008 tienen bloqueo de hito contradictorio entre fuentes:** el cuerpo narrativo del LaTeX dice que ADR-002 (Framework UI) bloquea el **Hito 2**, pero la tabla de ADRs del mismo LaTeX dice que bloquea el **Hito 4** — y el Hito 4 es, de hecho, donde se implementa la UI (US-401), lo cual sugiere que la tabla tiene razón y el texto narrativo del Hito 2 tiene un error de copy-paste. Mismo patrón con ADR-008 (Backup): el texto dice "Hito 3", la tabla dice "Hito 4", y el Hito 4 es donde vive US-402 (Backup) — la tabla parece ser la correcta en ambos casos. Se documenta la discrepancia tal cual en la sección 18 en vez de resolverla unilateralmente, para que el equipo lo confirme.
- **El estándar de análisis estático documentado no coincide con el real:** el Acuerdo de Ingeniería original (sección 5) decía `flake8/pylint`; el proyecto usa `ruff` en la práctica (`pyproject.toml`, CI). Ya corregido en la transcripción.
- **La tabla "Estructura de Repositorios"** (sección 17) le faltaba la fila de Competencia en ambas fuentes originales — es la inconsistencia que dio origen a esta revisión. Corregida acá.

Lo que ya está sólido (para no perder de vista en medio de tanto hallazgo)

- ☒ `SQLiteManager` (`database_manager.py`): conexión, `PRAGMA foreign_keys`, `row_factory`, inicialización de schema/vistas/seed/limpieza, con manejo de errores y logging. 15 tests de integración reales.
- ☒ Las 4 vistas SQL (`views.sql`) funcionan y están probadas, incluyendo protección contra división por cero.
- ☒ `SqliteUsuarioRepositorio` funcional, probado, y con el mapeo `pw↔contrasenia` correctamente resuelto — el único repositorio con tests dedicados hoy.

- ☒ `SqliteClubRepositorio` funcional (patrón de conexión directa, igual que los otros 4).
- ☒ `SqliteCompetenciaRepositorio` ya existe (ya no es el gap original) — 10 de sus 12 métodos tienen implementación real, aunque con los bugs puntuales ya detallados arriba.
- ☒ Pipeline CI real con dos workflows (`linter.yml`, `test.yml`) corriendo en cada PR — ver [docs/ideas-aprendizaje.md](#) sección 7 para el detalle de qué le falta para ser "completo".
- ☒ Suite de tests dividida por repositorio (`test_repositorios_*.py`), más fácil de mantener que el archivo único anterior — aunque dos de esos archivos todavía están vacíos (ver arriba).